Meeting 12. Big O and the Machine

Opening The Memory of a Computer · Recap 1 What Big O Counts · Recap 2 What Big O Leaves Out · A deeper look 1 Memory and Computation · A deeper look 2 What Computing Costs · A deeper look 3 Hardware Built for the Product · Closing Back to the O

Meeting 11 applied a layer of neurons to data with `X @ W.T + b` in NumPy, PyTorch, and on a GPU. Meeting 12 examines the cost of that computation. The operation **count**, described by O( ) notation, gives its growth with input size. Running time also depends on the implementation and hardware: Python overhead, memory access order, cache behaviour, and parallel execution. The notebooks measure these effects and use the results to estimate running times.

Every group did the two notebooks before the meeting of Monday, October 12, posted its results on the board, and every student uploaded both notebooks. The meeting opened with the cache experiment, the memory hierarchy and the measured bandwidth of each level (the opening deck); two **recaps** then went over the two notebooks, the operation count and what it leaves out; a **deeper look** returned to memory for operations per byte, the roofline, blocking and the transfers to a GPU, then went to what computing costs and to the hardware built for the product; and a short closing, *Back to the O*, returned to the exponent of matrix multiplication. These notes state the main points first, then walk through both notebooks part by part, then develop the three deeper topics. Code and output use group 7's example, the image *The Bedroom* by Vincent van Gogh; the slides use the same example. The notes were built on a machine without a GPU; every number of the T4 is the class's own, from the posts of meeting 11, and the figures of data sheets and papers are marked as such. There is no recording of the meeting: the boxes *In the meeting* say what the decks presented and what the class had posted, not what was said in the room.

The notebooks and the pages of this meeting.

Each notebook opens in Google Colab. Choose *Save a copy in Drive* in the *File* menu before you change anything, to save your work.

- The first notebook: before the meeting (https://colab.research.google.com/github/ikoutis/cs301-labs/blob/main/unit-2/notebooks/meeting-12-before-class.ipynb)
  Parts 1 to 6: the layer on a GPU, the product on both processors, the count, operations per second, where the numbers are, a prediction. It asks Colab for a GPU.
- The second notebook: rows and columns (https://colab.research.google.com/github/ikoutis/cs301-labs/blob/main/unit-2/notebooks/meeting-12-rows-and-columns.ipynb)
  Parts 1 to 6: a matrix as one line of numbers, loops in both orders, NumPy inside the loop, a power of two, column sums read by rows, what NumPy does by itself. No GPU.
- CPU and GPU (https://ikoutis.github.io/cs301-labs/unit-2/pages/cpu-and-gpu.html) the two processors, what is inside each of them, and how one product is divided among thousands of cores
- What matrix operations cost (https://ikoutis.github.io/cs301-labs/unit-2/pages/matrix-costs.html) the count of every operation of the unit, and the notation O( )

The seven decks of the meeting, in the order they were shown; each is a web page with the narration in its notes, and a PDF.

- The Memory of a Computer (https://ikoutis.github.io/cs301-labs/unit-2/slides/m12-opening.html) the opening (PDF)
- What Big O Counts (https://ikoutis.github.io/cs301-labs/unit-2/slides/m12-recap-1.html) Recap 1 (PDF)
- What Big O Leaves Out (https://ikoutis.github.io/cs301-labs/unit-2/slides/m12-recap-2.html) Recap 2 (PDF)
- Memory and Computation (https://ikoutis.github.io/cs301-labs/unit-2/slides/m12-deeper-1.html) A deeper look 1 (PDF)
- What Computing Costs (https://ikoutis.github.io/cs301-labs/unit-2/slides/m12-deeper-2.html) A deeper look 2 (PDF)
- Hardware Built for the Product (https://ikoutis.github.io/cs301-labs/unit-2/slides/m12-deeper-3.html) A deeper look 3 (PDF)
- Back to the O (https://ikoutis.github.io/cs301-labs/unit-2/slides/m12-back-to-o.html) the closing (PDF)

The notebooks are the reference for Review Quiz 12, whose questions name their parts.

The main points

The following ten ideas summarize the meeting. Each links to the relevant explanation and code.

1 A layer costs n × K × m passes, and O(nKm) says how that grows

A **neuron** is a rule that converts a point into one number, its score: it multiplies each coordinate of the point by a weight, adds the products, and adds a number of its own, the bias. A **layer** is a set of K neurons that read the same point, and on n points with m coordinates its scores are the matrix `X @ W.T + b`. Written with three loops, one line does the work, `total += W[j, c] * X[i, c]`, and the loops pass through it once for every point, every neuron and every coordinate: $n \times K \times m$ passes, two arithmetic operations each. The time of the loops grows as the count does, roughly: in the measurement below, doubling n multiplied the time by 1.9 and doubling all three numbers multiplied it by 7.9, for the count's 2 and 8, and the time of one pass, in the last column, moved far less than the times did. Timing varies between runs. The operation-count ratios are exactly 2 and 8; elapsed-time ratios should be compared with those predictions. Computer science writes this as **O(nKm)**, "of the order of nKm": Big O gives an asymptotic upper bound on growth, omitting constant factors and lower-order terms. The exact iteration count here is nKm; its conversion to time depends on measured throughput. For $n = m = K$ the count is the **cube** of the size, eight times the work for twice the size. A cube grows fast, but it is not an exponential: 2^{size} doubles each time the size grows by one; the cube is multiplied by 8 each time the size doubles.

OUTPUT

```
     n    m    K   passes, n x K x m   seconds   ratio to the first   nanoseconds per
pass
   400   30   20             240,000     0.043                  1.0
178
   800   30   20             480,000     0.083                  1.9
172
   800   60   40           1,920,000     0.338                  7.9
176
```

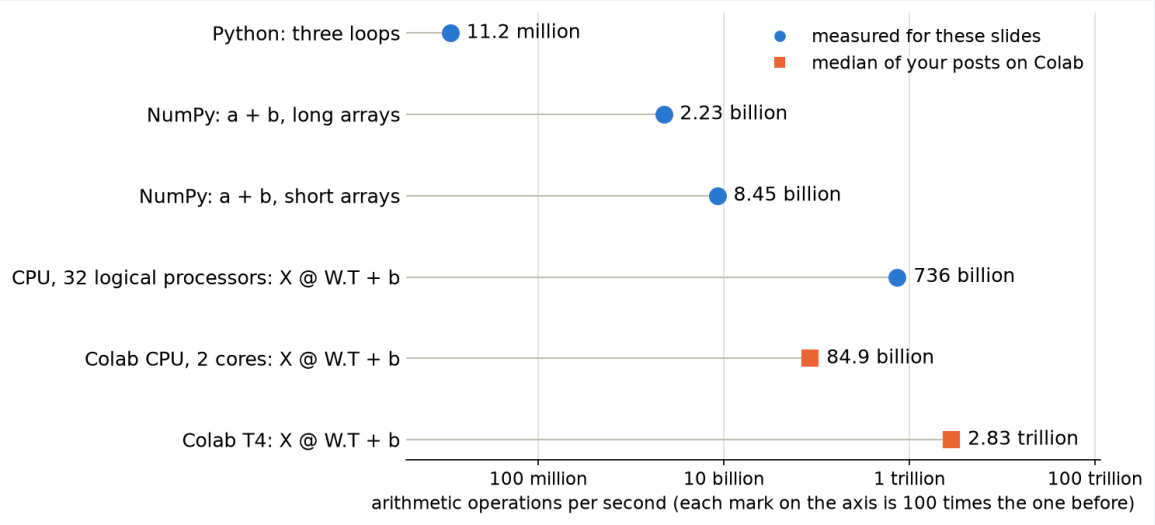
Section 1.3. The page What matrix operations cost lists the count of every operation of the unit.

2 Operations per second and timing estimates

To estimate elapsed time from an operation count, measure the rate in **operations per second**, each addition and each multiplication counting as one. The chart compares six rates: three loops in Python, about 11.2 million operations a second on the machine that built these notes; NumPy adding two long arrays and two short arrays; the product

`X @ W.T + b` at size 4,096 on that machine (32 logical processors); and, from the groups' posts, the same product on Colab's CPU with two cores, 84.9 billion operations a second, and on its T4, 2.83 trillion. The rates span millions to trillions of operations per second. A count and a measured rate give a time estimate,

time ≈ operations / operations per second, which the last part of the first notebook tests: each group chose a layer, predicted its time on the CPU and on the GPU, and measured it.

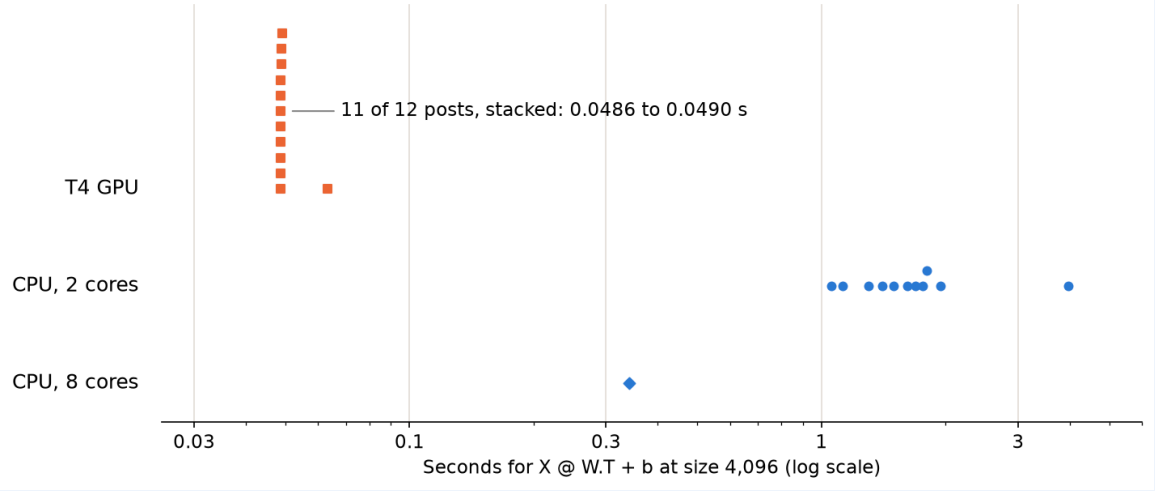


Six rates of arithmetic: four measured on the machine that built these notes (blue), two the medians of the class's posts on Colab (orange).

Sections 1.4 and 1.6.

3 Your T4: the class's own measurement of a GPU

Twelve groups measured the product at size 4,096 on a Colab T4 after meeting 11 and posted the line. The count was the same in every post, 137 billion operations. The T4 took 0.0486 to 0.0490 seconds in eleven of the twelve posts and 0.0631 in the twelfth; Colab's CPU with two cores took 1.06 to 1.95 seconds, one session 3.97. The GPU was 22 to 40 times as fast in ten of those posts and 63 times in one; the median factor was 33. One session had a CPU with eight cores: 0.342 seconds, and a factor of 7. A GPU is faster at this product because thousands of small cores work on it at once and the product divides among them; the page CPU and GPU shows how. Using the GPU also requires copying data into its memory across a link with lower bandwidth than either processor's memory.

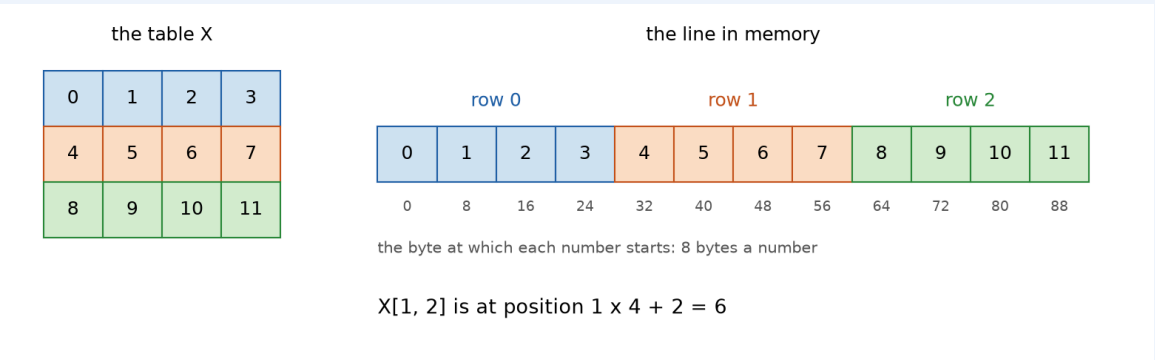


The product at size 4,096, from the twelve posts of the meeting 11 board that measured a T4.

Section 1.2.

4 A matrix is one line of numbers

The memory of a computer is one long line of bytes, each with an address. The contiguous NumPy arrays used here store their entries in consecutive memory locations: the rows one after another, **row order**, so that the number in row i and column j of a matrix with m columns is at position $i \times m + j$ of the line. The **strides** say how many bytes to step for one move along each dimension: 8 to the next column, $8m$ to the next row, for numbers of 8 bytes. The transpose `X.T` moves no number: it swaps the two strides and reads the same line the other way. Other languages store the columns one after another, **column order**. Shape and strides describe the array's layout; access order affects performance.

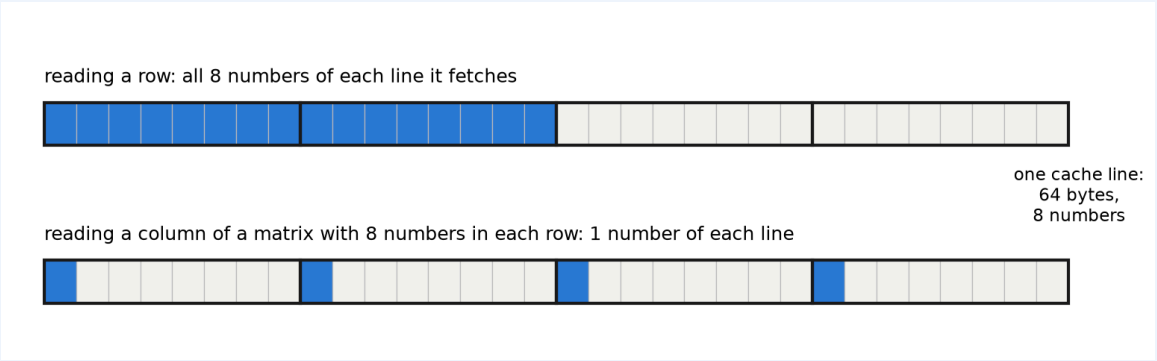


A matrix of 3 by 4, and the line in which NumPy stores it.

Section 2.1.

5 A cache line: why a row is cheaper to read than a column

Data moves from memory into the **cache**, a small fast memory near the processor, in **cache lines**. A 64-byte line holds eight adjacent 8-byte numbers. Reading a row uses all eight numbers of every line it fetches, and sequential access allows the processor to prefetch later lines. Reading a column uses one number of each line and jumps a whole row ahead for the next. While the matrix fits in the cache, the other seven numbers are still there when the next column needs them; when it does not fit, those entries may be evicted before reuse, requiring further reads from main memory. On the machine that built these notes, a loop that hands each row of a matrix to NumPy and a loop that hands each column cost the same for a matrix of half a megabyte (the columns took 1.1 times as long as the rows) and the columns took 19.1 times as long for a matrix of 134 megabytes. In loops written in Python alone the order shows little: interpreter overhead masks much of the difference in memory-access cost.

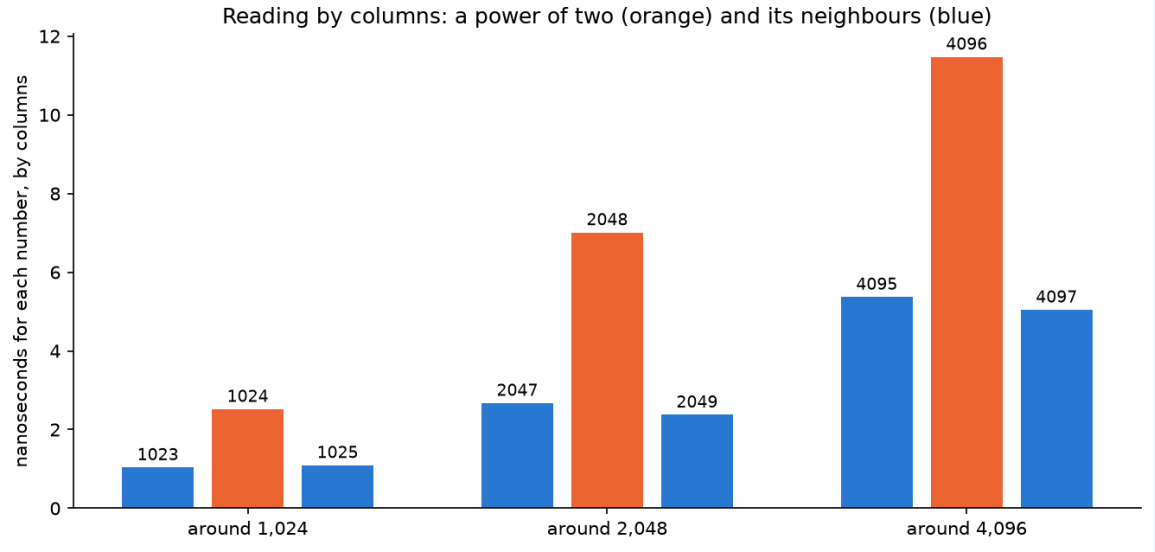


Reading a row uses every number of the lines it fetches; reading a column of a matrix with eight numbers in each row uses one.

Sections 2.2 and 2.3.

6 A power of two, and one more

A cache maps memory addresses to **sets**; the address of a line decides its set, and each set holds a limited number of lines. Lines that follow each other in memory go to sets that follow each other, so a row read in order spreads over all the sets. The numbers of a column are a whole row apart, and when a row is a power of two bytes long they all land in the same few sets and push each other out: lines can be evicted before the next column needs them, even when other cache sets have unused space. So on this machine a matrix with 2,048 columns, read by columns, cost 7.01 nanoseconds for each number, and a matrix with 2,049 columns cost 2.38: the larger matrix is the faster one. The fix is **padding**, rows stored one number longer with the last number unused, which linear algebra libraries can apply internally.

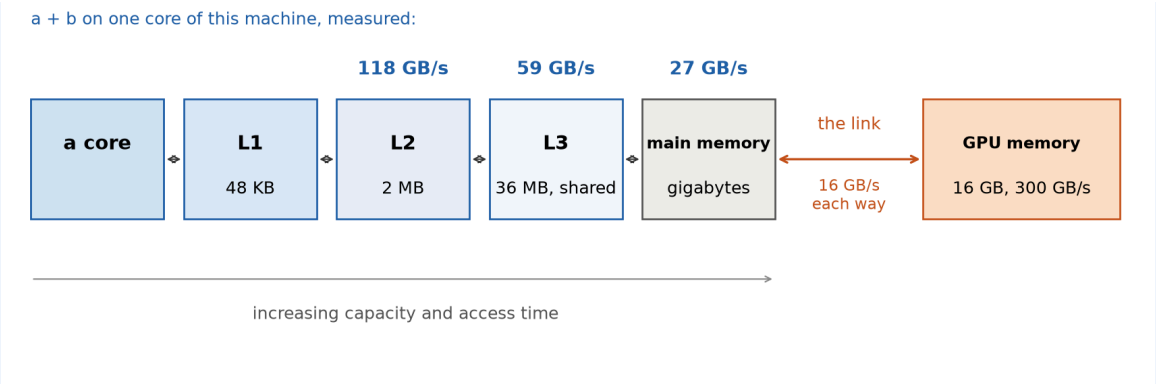


Reading by columns around 1,024, 2,048 and 4,096: the power of two (orange) and its two neighbours (blue), on the machine that built these notes.

Section 2.4.

7 Where the numbers are: a hierarchy of memories

A processor keeps the numbers it works on in a hierarchy of memories: the levels of its cache, **L1**, tens of kilobytes beside each core, **L2**, about a megabyte, and **L3**, tens of megabytes shared by the cores; behind them the **main memory** of the computer, gigabytes, and slower to reach; and, across a link, the GPU's own memory. The same addition, `a + b`, on arrays of sixteen lengths shows changes in speed as the arrays outgrow the caches. On the machine of these notes it ran at about 9.81 billion additions a second while the three arrays fitted in L2, 4.94 billion in L3, and 2.28 billion from the main memory. Each addition moves 12 bytes, two numbers in and one out, so these rates estimate the **effective bandwidth** of each level, the bytes it delivers in a second: 118 billion bytes a second from L2 and 27.4 billion from the main memory, for one core. The T4's memory delivers 300 billion bytes a second by its data sheet, and the link that joins it to the computer's memory 16 billion each way.

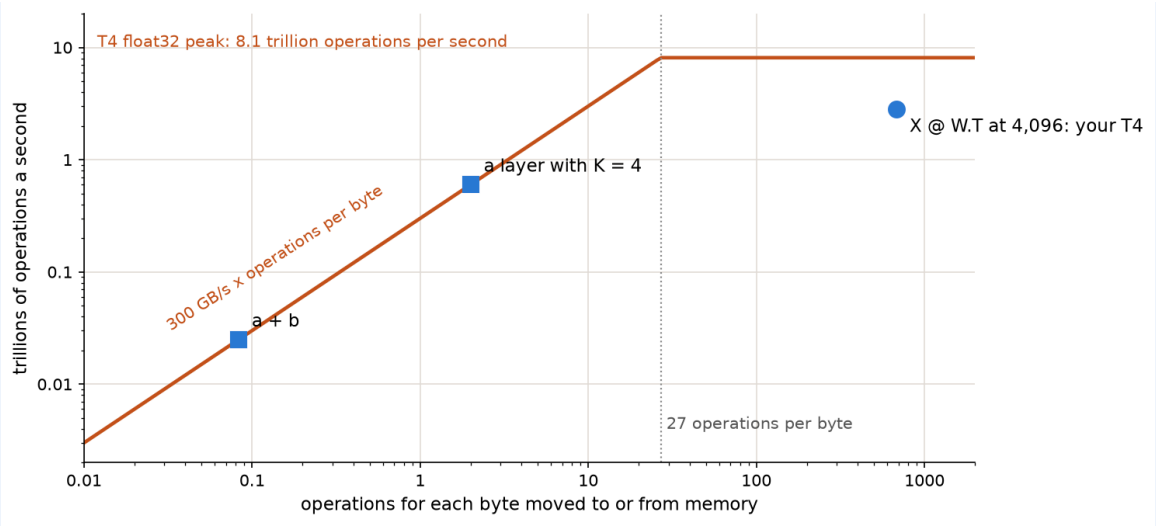


The hierarchy of memories of the machine that built these notes, and the T4's memory across the link; the bandwidths above the boxes were measured with a + b on one core.

Sections 1.5 and 3.1.

8 Memory-bound and compute-bound: operations per byte, and the roofline

Whether a computation waits for its numbers or for its arithmetic depends on how many operations it does for each byte it moves. `a + b` does one addition for 12 bytes and is **memory-bound**: the bandwidth sets its speed, and on the T4 it could never exceed 25 billion additions a second, under the stated bandwidth limit. The product of two matrices of size N does $2N^3$ operations on $12N^2$ bytes, $N\,/\,6$ operations for each byte, 683 at $N = 4,096$, in the idealized case where each fetched number is reused N times; it is **compute-bound**, and the arithmetic sets its speed. The **roofline** draws the two limits of one processor on one chart, its peak rate and its bandwidth times the operations per byte, and a computation sits under the lower of the two. Under the T4's roof the class's product sits at 2.8 trillion operations a second, a third of the peak of the data sheet.



The roofline of the T4, from its data sheet, with a + b, a layer of four neurons and the class's product under it.

Sections 1.4, 3.2 and 3.3.

9 The same O( ), different seconds

The table compares implementations, data layouts, and processors. Similar operation counts can produce very different times or rates. The approximate factors come from these notes' measurements and the class posts; compare them with your own notebook results.

What O( ) leaves out	Measured before the meeting	How much
the interpreter	the loops of Python against NumPy	about 100 times
the order of reading	a matrix by rows or by columns	1 to about 20 times
the cache's sets	2,048 columns against 2,049	2 to 4 times
where the numbers are	`a + b` in the cache against the main memory	about 5 times
operations per byte, and the cores	`a + b` against the product, on one CPU	hundreds of times
the processor	the T4 against Colab's CPU, in the posts	22 to 40 times

Estimate running time using both operation counts and measured rates.

The whole walkthrough; the table closes Recap 2.

10 What computing costs: joules and dollars

A computation has three costs. Its time is the count divided by the rate. Its **energy** is the power that the processor draws, in **watts**, joules each second, multiplied by the time; electricity is sold by the kilowatt-hour, 3.6 million joules. Its price is what an hour of the processor costs, multiplied by the hours. At the T4's rated 70 watts, one product at size 4,096 takes at most 3.4 joules; a T4 rented from Google Cloud at 35 cents an hour does a trillion operations for about 0.0034 cents of rent, and the electricity is a small part of that. Training a large model requires many such operations: GPT-3 took 3.14×10^{23} operations, ten thousand GPUs for two weeks and 1,287 megawatt-hours, and at the class's rate one T4 would need about 3,526 years. Combining these factors gives $\text{cost} \approx \text{count} \times \text{seconds for one operation} \times \text{price of a second}$: a faster processor can cost more per hour yet less per operation, depending on its price and the workload's achieved throughput.

Sections 4.1 to 4.5.

Working with the notebooks, and with an AI assistant.

Each notebook is run from the top, one cell at a time, with Shift+Enter. A paragraph that begins with **Your turn** asks for something before you go on, and a cell under it checks what you wrote and says what to look at if something is wrong. Two parts, Part 5 of each notebook, ask an AI assistant to write a function to stated requirements, and the prompt you used goes into the cell marked **Your prompt**: a precise prompt is part of the work, and it states the name of the function, its arguments, what it may use and what it may not, and what it returns. The cells marked **Your thinking** ask for a few sentences in your own words on what you measured; the upload is graded on the prompt and thinking cells, eleven in all. Keep your measurements and compare the ratios, allowing for differences between machines.

The first notebook. What Big O counts

Open the first notebook (https://colab.research.google.com/github/ikoutis/cs301-labs/blob/main/unit-2/notebooks/meeting-12-before-class.ipynb)

1.1 The layer of meeting 11, on a GPU Part 1

The notebook begins by making sure that the session has a GPU: it asks Colab for one, and its first cell says whether PyTorch sees it. The layer it builds is the one of meeting 11. The points are the pixels of the group's image, n of them with $m = 3$ coordinates, red, green and blue. The five neurons stand for five colours that occur often in the image, found by a few rounds of a method that gathers the pixels around K centres; the weights of neuron j are its colour, and its bias is minus half the sum of the squares of that colour, so that the largest score of a pixel belongs to the colour nearest to it. The scores are computed with PyTorch, whose arrays are called **tensors**, in numbers of 4 bytes, **float32**, on the CPU, and every pixel is drawn in the colour of its neuron: image 7 in five flat colours.

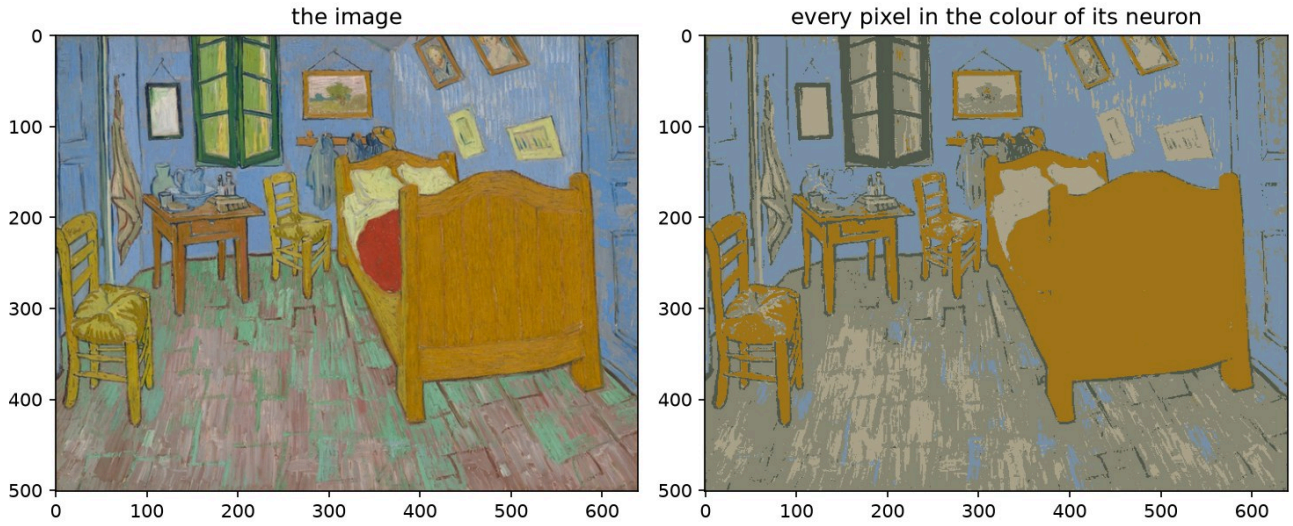


Image 7, and every pixel in the colour of its neuron.

A tensor is stored on a **device**: in the memory of the CPU, or in the memory of the GPU, which PyTorch calls `"cuda"`. `X_t.to(device)` returns a tensor on the requested device, copying the data if the device changes and returning the original tensor if it is already there, and an operation needs all of its tensors on one device, whose processor then carries it out. The part asks for the function `scores_on(device)`: three tensors on the requested device, then the scores computed using those tensors. The check runs it and names the mistake it finds: a tensor left on the CPU while the others are on the GPU, a missing transpose, a missing bias. The question asks why the operands must be on the same device and when a transfer is needed. In this setup the GPU uses tensors in its own memory, so the inputs must be transferred there. The resulting scores are also stored there. `.cpu()`

copies them back, which the check needs in order to compare them with the scores the CPU computed.

Code: Part 1, the layer, the function scores_on, and its check — cells `part1-layer`, `paste-device`, `part1-check`

```
X = image.reshape(-1, 3)                # one row for each pixel: n points with m = 3 coordinates
n, m = X.shape
K = 5
C = reference_colours(X, K)             # K rows and 3 columns: five colours of the image, darkest first

X_t = torch.from_numpy(X).float()       # the points, as a tensor of float32 numbers
W_t = torch.from_numpy(C).float()       # the weights of the layer: row j is the colour of neuron j
b_t = -0.5 * (W_t ** 2).sum(dim=1)      # the biases: minus half the sum of the squares of each colour
S_t = X_t @ W_t.T + b_t                 # the scores, computed by the CPU: n rows and K columns
print(f"X_t: {tuple(X_t.shape)}   W_t: {tuple(W_t.shape)}   b_t: {tuple(b_t.shape)}   S_t:
{tuple(S_t.shape)}   device: {S_t.device}")

winner = S_t.argmax(dim=1).numpy()      # for each pixel, the neuron with the largest score
show([image, C[winner].reshape(height, width, 3)], ["the image", "every pixel in the colour of its
neuron"])
```

OUTPUT

```
X_t: (320640, 3)   W_t: (5, 3)   b_t: (5,)   S_t: (320640, 5)   device: cpu
```

```
device = "cuda" if GPU else "cpu"       # the GPU, when this session has one


def scores_on(device):
    """Copy the points, the weights and the biases to the device, and compute the scores of the layer
there."""
    X_d = X_t.to(device)
    W_d = W_t.to(device)
    b_d = b_t.to(device)
    return X_d @ W_d.T + b_d
```

OUTPUT

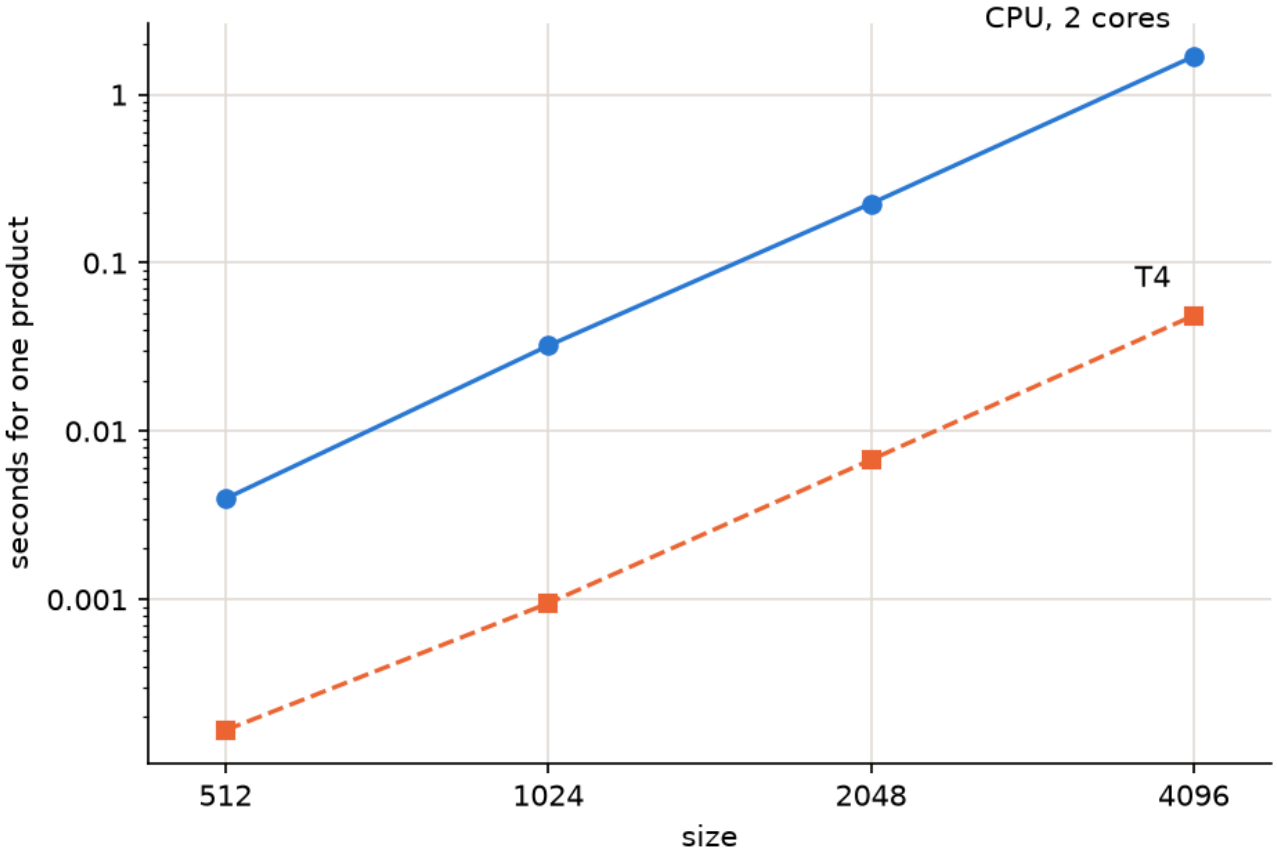
```
Correct: the scores were computed on the device 'cpu', and .cpu() copies them back. The largest
difference from the CPU's scores: 0.0e+00.
```

1.2 The product on both processors Part 2

The layer of Part 1 is a small product: five neurons, three coordinates. The layers of real networks are far larger, so the part measures `X @ W.T + b` with n, m and K all equal to a size, at four sizes from 512 to 4,096, on the CPU and, when the session has one, on the GPU. Two details are particular to a GPU. PyTorch hands a computation to the GPU and goes on at once, before the result is ready, so `torch.cuda.synchronize()` waits for the GPU before the clock is read. And the first use of a device takes extra time, so one small product is run first and not measured. The cell also times the copy of one matrix of size 4,096, 67 megabytes, into the memory of the GPU. The

table and the chart hold the same numbers; both axes are logarithmic, so cubic growth appears as a straight line. The part then asks for two numbers read from the table: by what factor the GPU is faster at size 4,096, and by what factor the CPU's time grows from 2,048 to 4,096, which Part 3 explains.

The machine that built these notes has no GPU, so its table, in the accordion below, has one column. The class's own measurements are in the main points: the dot plot of the twelve posts in idea 3, and below, the one group that posted its whole table. From 2,048 to 4,096 its CPU's time grew by 7.5 and its T4's by 7.2, the count's 8 on both processors, and the two lines are nearly parallel: the T4 was 24 times as fast at size 512 and 35 times at 4,096. O( ) describes the growth, the slope of the lines, and says nothing about the distance between them.



One group's whole table: Colab's CPU with two cores, and its T4, at the four sizes.

Why the GPU is faster is the subject of the page CPU and GPU: thousands of small cores, and a product that divides into pieces they can do at the same time. The exercise also asks about copying cost. A matrix of 67 megabytes crosses the link to the GPU in a few milliseconds, against about fifty for the product at that size; a program that copies its data to the GPU for one small operation and copies the result back would spend more time on the copies than on the work. Section 3.5 returns to this with the numbers of the data sheet.

Code: Part 2, the measurement, the two answers, and the check

cells `part2-time`, `part2-chart`, `answers-table`, `part2-check`

```
def finish(device):
    """Wait until the device has finished its work. A GPU may still be working when PyTorch goes on."""
    if device == "cuda":
        torch.cuda.synchronize()


def time_product(size, device):
    """The average time, in seconds, of X @ W.T + b on the device, with n = m = K = size."""
    X = torch.rand(size, size, device=device)        # random float32 numbers, made on the device
    W = torch.rand(size, size, device=device)
    b = torch.rand(size, device=device)
    repeats = max(1, min(200, 8_000_000_000 // size ** 3))     # many repeats when the product is small
    finish(device)
    start = time.perf_counter()
    for _ in range(repeats):
        S = X @ W.T + b
    finish(device)
    return (time.perf_counter() - start) / repeats


sizes = [512, 1024, 2048, 4096]
devices = ["cpu", "cuda"] if GPU else ["cpu"]
seconds = {}
for d in devices:
    time_product(64, d)                  # one small product that is not measured: the first use of a
device is slow
    seconds[d] = [time_product(size, d) for size in sizes]

gpu_name = torch.cuda.get_device_name(0) if GPU else "none"
print(f"X @ W.T + b with n = m = K = size, in seconds. CPU: {os.cpu_count()} logical cores. GPU:
{gpu_name}.")
print(f"{'size':>6s}{'multiplications':>18s}{'CPU':>12s}" + (f"{'GPU':>12s}{'CPU / GPU':>12s}" if GPU else
""))
for i, size in enumerate(sizes):
    line = f"{size:6d}{size ** 3:18,d}{seconds['cpu'][i]:12.6f}"
    if GPU:
        line += f"{seconds['cuda'][i]:12.6f}{seconds['cpu'][i] / seconds['cuda'][i]:12.1f}"
    print(line)

if GPU:
    matrix = torch.rand(4096, 4096)                  # one matrix in the memory of the CPU: 4096 x 4096
float32 numbers
    finish("cuda")
    start = time.perf_counter()
    copy = matrix.to("cuda")                         # its copy in the memory of the GPU
    finish("cuda")
    t_copy = time.perf_counter() - start
    megabytes = matrix.element_size() * matrix.nelement() / 1e6
    print(f"\nCopying one matrix of size 4096 ({megabytes:.0f} MB) to the GPU took {t_copy:.4f} seconds; "
          f"the product at that size took the GPU {seconds['cuda'][-1]:.4f} seconds.")
```

OUTPUT

```
X @ W.T + b with n = m = K = size, in seconds. CPU: 32 logical cores. GPU: none.
  size   multiplications         CPU
   512      134,217,728    0.000641
  1024    1,073,741,824    0.003525
  2048    8,589,934,592    0.027090
  4096   68,719,476,736    0.186800
```

OUTPUT

```
Line for the discussion board:
No GPU in this session. CPU with 32 logical cores. Size 4096: 0.187 s. Size 512: 0.6
milliseconds.
```

```
GPU_FACTOR = seconds["cpu"][-1] / seconds["cuda"][-1] if GPU else None
CPU_GROWTH = seconds["cpu"][-1] / seconds["cpu"][-2]
```

OUTPUT

```
Correct: from size 2,048 to 4,096 the time of the CPU grows by a factor of 6.9.
```

1.3 Count the operations Part 3

The scores of a layer, computed with three loops: for every point, for every neuron, for every coordinate, `total += W[j, c] * X[i, c]`, one multiplication and one addition. The loops pass through that line $n \times K \times m$ times. The part measures them for three layers: the first with $n = 400$ points, $m = 30$ coordinates and $K = 20$ neurons; the second with n doubled; the third with all three doubled. Before the measurement it asks for a prediction from the count: the passes grow by a factor of 2 from the first layer to the second, and by $2 \times 2 \times 2 = 8$ from the first to the third. The check rejects 4 for the first factor, since only n changes, and 6 for the second, since three doublings multiply rather than add.

The measured times approximately follow the counts. Other activity affects short timings, but the time per pass remains within a similar range across the three rows. This supports the estimate $\text{time} \approx (\text{time of one pass}) \times n \times K \times m$: on this machine 178 nanoseconds a pass in the first row. Computer science writes this as $\mathbf{O(nKm)}$. The notation keeps how the number of operations grows with the sizes and drops the constant in front. For a layer with $n = m = K$ the count is the cube of the size, giving eight times the work when the size doubles, which predicts the Part 2 timing ratio only if throughput remains similar. The cube is not an *exponential*: 2^{size} is multiplied by 2 each time the size grows by one, and at size 512 it already has 155 digits. The question of the part, a layer of $K = 512$ neurons on a batch of $n = 256$ images, first asks how the count changes: with the batch twice as large and the layer half as large, $2n \times K \,/\, 2 \times m$ gives the same operation count and a similar predicted time; with n, m and K all doubled, eight times the arithmetic work and a predicted eightfold time increase at similar throughput.

Code: Part 3, the prediction, its check, and the three loops timed — cells `predict-count`, `part3-check`, `count`

```
FACTOR_N_DOUBLED = 2
FACTOR_ALL_DOUBLED = 8
```

OUTPUT

```
Correct: 240,000, 480,000 and 1,920,000 passes. Now run the next cell and compare the times.
```

```
def layer_loops(X, W, b):
    """The scores of the K neurons of a layer on the n points of X, one multiplication at a time."""
    n, m = X.shape
    K = W.shape[0]
    S = np.empty((n, K))
    for i in range(n):                  # every point
        for j in range(K):              # every neuron
            total = b[j]
            for c in range(m):          # every coordinate
                total += W[j, c] * X[i, c]
            S[i, j] = total
    return S


layers = [(400, 30, 20), (800, 30, 20), (800, 60, 40)]      # n, m and K of the three layers
rng = np.random.default_rng(0)          # a source of random numbers
loop_times = []
print(f"{'n':>6s}{'m':>5s}{'K':>5s}{'passes, n x K x m':>20s}{'seconds':>10s}{'ratio to the first':>21s}
{'nanoseconds per pass':>23s}")
for n_, m_, K_ in layers:
    X_, W_, b_ = rng.random((n_, m_)), rng.random((K_, m_)), rng.random(K_)
    start = time.perf_counter()
    layer_loops(X_, W_, b_)
    loop_times.append(time.perf_counter() - start)
    print(f"{n_:6d}{m_:5d}{K_:5d}{n_ * K_ * m_:20,d}{loop_times[-1]:10.3f}{loop_times[-1] /
loop_times[0]:21.1f}"
          f"{loop_times[-1] / (n_ * K_ * m_) * 1e9:23.0f}")
```

OUTPUT

n	m	K	passes, n x K x m	seconds	ratio to the first	nanoseconds per pass
400	30	20	240,000	0.043	1.0	178
800	30	20	480,000	0.083	1.9	172
800	60	40	1,920,000	0.338	7.9	176

1.4 Measured operations per second Part 4

The count does not say how long one operation takes. That is measured as the number of **arithmetic operations per second**, the operations divided by the seconds, each multiplication and each addition counting as one. The part asks for the function `product_rate(size, seconds)`: a product with $n = m = K = \text{size}$ has size^3 passes of two operations, so the rate is $2 \times \text{size}^3 \,/\, \text{seconds}$.

The check names the three usual mistakes: one operation for each pass, the square of the size in place of the cube, and the division the wrong way round.

The experiment compares five arithmetic rates, counting each addition and multiplication as one operation. The three loops of Python, from Part 3: about 11.2 million operations a second here. NumPy adding two arrays of about 16 million numbers, four times: 2.23 billion a second. NumPy adding two arrays of about 16 thousand numbers, 4,096 times, which is the same number of additions by the same code: 8.45 billion a second. The product at size 4,096 on the CPU, from the table of Part 2: 736 billion a second on this machine (32 logical processors), 65,570 times the loops. And the same product on the GPU, where the session has one: 2.8 trillion a second on the class's T4, from the posts. The chart with the class's rows is in idea 2. Four factors explain the rate differences.

- **Interpreter overhead.** The loops run in the interpreter of Python, which does a great deal of work around each operation; NumPy's loops run in code made for the processor.
- **Array size and cache capacity.** The code performs the same additions, but the arrays may be served from different memory levels. The processor has a small fast memory of its own, the **cache**. The short arrays fit into it and stay there. The long arrays do not fit: each number is fetched from the main memory, used in one addition, and the result is sent back, and the processor spends most of its time waiting. Such a computation is **memory-bound**: the memory sets its speed, not the arithmetic.
- **The product.** In `X @ W.T`, each entry of `X` is multiplied by a weight from every neuron. The product is programmed to work on blocks of the two matrices that fit in the cache, and to use each number many times while it is there (section 3.4). The processor spends its time on arithmetic: the computation is **compute-bound**.
- **The GPU.** Thousands of small cores work on the same product. The data sheet of the T4 gives 8.1 trillion operations a second at its peak; the class reached a third of that, as section 3.3 discusses.

O( ) describes operation-count growth. The measured rates also depend on how the program is run, where the numbers are, and which processor does the work.

Code: Part 4, the rate of a product, its check, and the five ways of computing — cells `paste-rate`, `part4-check`, `rates`

```
def product_rate(size, seconds):
    """Arithmetic operations per second of the product X @ W.T with n = m = K = size, which took
`seconds`."""
    return 2 * size ** 3 / seconds
```

OUTPUT

```
Correct: the product at size 4,096 in 0.05 seconds is 2.75 trillion operations per second.
```

```
rng = np.random.default_rng(1)
rates = {}                              # for each way of computing: arithmetic operations per second
```

```
# 1. Python, three loops: one multiplication and one addition for each pass
n_, m_, K_ = layers[0]
rates["Python: three loops"] = 2 * n_ * K_ * m_ / loop_times[0]

# 2. NumPy, a + b on long arrays: one addition for each pair of numbers
long = 2 ** 24                          # 16,777,216 numbers in each array
a = rng.random(long, dtype=np.float32)
b = rng.random(long, dtype=np.float32)
c = np.empty(long, dtype=np.float32)
np.add(a, b, out=c)                     # c = a + b, with the result written into the array c; not
measured
start = time.perf_counter()
for _ in range(4):
    np.add(a, b, out=c)
rates["NumPy: a + b, long arrays"] = 4 * long / (time.perf_counter() - start)

# 3. NumPy, a + b on short arrays: the same number of additions
short = 2 ** 14                         # 16,384 numbers in each array
a, b, c = a[:short].copy(), b[:short].copy(), c[:short].copy()
calls = 4 * long // short
np.add(a, b, out=c)
start = time.perf_counter()
for _ in range(calls):
    np.add(a, b, out=c)
rates["NumPy: a + b, short arrays"] = calls * short / (time.perf_counter() - start)

# 4 and 5. The product at size 4,096, from Part 2. The bias adds 4,096 x 4,096 additions, too few to
count.
rates["CPU: X @ W.T + b"] = product_rate(4096, seconds["cpu"][-1])
if GPU:
    rates["GPU: X @ W.T + b"] = product_rate(4096, seconds["cuda"][-1])

slowest = min(rates.values())
print(f"{'way of computing':30s}{'operations per second':>24s}{'ratio to the slowest':>23s}")
for name, value in rates.items():
    print(f"{name:30s}{in_words(value):>24s}{value / slowest:23,.0f}")
```

OUTPUT

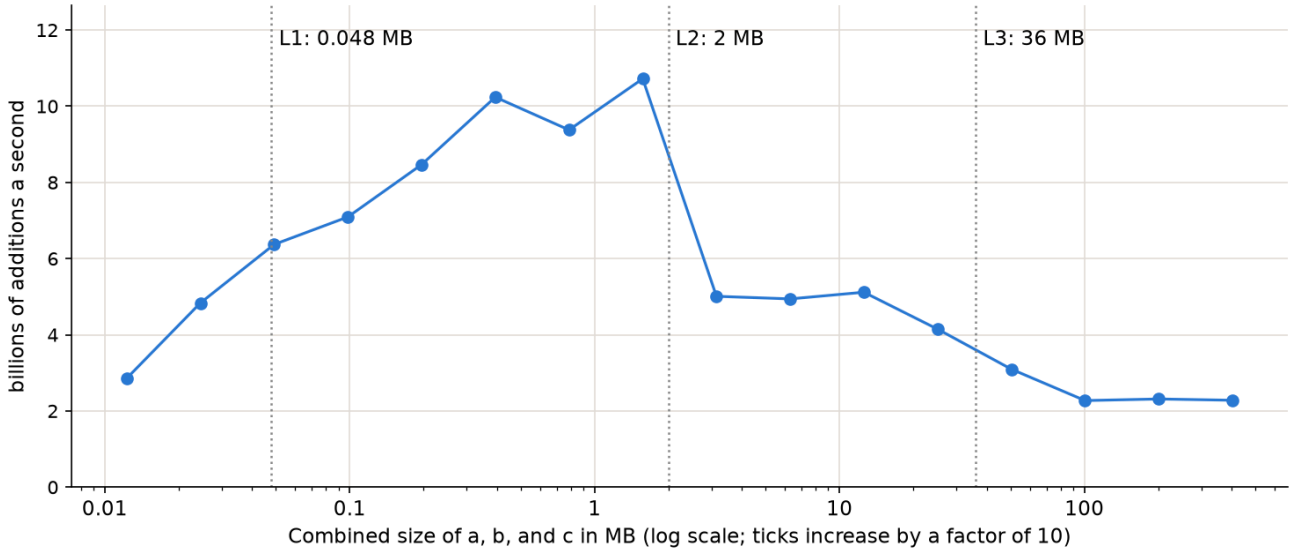
```
way of computing                operations per second   ratio to the slowest
Python: three loops                       11.2 million                      1
NumPy: a + b, long arrays                 2.23 billion                    199
NumPy: a + b, short arrays                8.45 billion                    753
CPU: X @ W.T + b                           736 billion                 65,570
```

1.5 Where the numbers are Part 5

Part 4 found that the same additions run faster on short arrays than on long ones. This part measures the rate of `a + b` at sixteen lengths, from about a thousand numbers to 33 million, about 67 million additions at each length, with a function that an AI assistant writes to five requirements: its name and arguments, `add_rate(length, repeats)`; two arrays of random float32 numbers and a third for the result; one addition not measured, then `repeats` of them timed with `time.perf_counter()`; the additions per second returned, `length × repeats` divided by the seconds;

and no loop over the numbers of the arrays. The check times the assistant's function against NumPy's addition timed the same way and names what it finds: seconds returned in place of a rate, the additions of one call only, or a loop over the numbers.

The chart below is the sweep as it was run for these notes, the best of five runs at each length, with the sizes of this machine's caches marked: 48 kilobytes of L1 beside each core, 2 megabytes of L2 for a core, 36 megabytes of L3 shared by all of them, by the maker's figures. The rate generally falls as the combined arrays outgrow the caches and rely on slower memory. The transitions need not form sharp steps. In Colab the notebook reads the sizes of the caches from the system and marks them itself; its processors have other sizes, and the groups' charts dropped elsewhere. At the far left of the chart another cost shows: each call of `np.add` takes a fixed time of its own, whatever the length, which is a significant fraction of the time for short arrays. The second half of the question of the part, why the product of Part 4 does not slow down in the same way when its matrices outgrow the cache, is section 3.4: the product is computed in blocks that stay in the cache while each of their numbers is used many times.



The rate of a + b against the size of the three arrays, on the machine that built these notes; the best of five runs at each length.

Code: Part 5, a function written to the requirements, its check, and the sweep — cells `paste-add-rate`, `part5-check`, `add-sweep`

```
def add_rate(length, repeats):
    """Additions per second of np.add on two arrays of `length` float32 numbers, measured over `repeats` calls."""
    rng = np.random.default_rng()
    a = rng.random(length, dtype=np.float32)
    b = rng.random(length, dtype=np.float32)
    c = np.empty(length, dtype=np.float32)
    np.add(a, b, out=c)                  # once, not measured
    start = time.perf_counter()
```

```
    for _ in range(repeats):
        np.add(a, b, out=c)
    seconds = time.perf_counter() - start
    return length * repeats / seconds
```

OUTPUT

```
Correct: add_rate gives 9.84 billion additions a second for two arrays of 16,384 numbers.
```

```
import glob

lengths = [2 ** k for k in range(10, 26)]                 # from 1,024 to 33,554,432 numbers in each array
sweep = [add_rate(length, max(1, 2 ** 26 // length)) for length in lengths]      # about 67 million
additions each
megabytes = [3 * 4 * length / 1e6 for length in lengths]   # a, b and c: three arrays of numbers of 4
bytes

caches = []                                                # the caches of this processor, where the
system tells them
for folder in sorted(glob.glob("/sys/devices/system/cpu/cpu0/cache/index*")):
    try:
        kind = open(folder + "/type").read().strip()
        level = open(folder + "/level").read().strip()
        size = open(folder + "/size").read().strip()
    except OSError:
        continue
    if kind != "Instruction":
        factor = {"K": 1e3 * 1.024, "M": 1e6 * 1.048576}.get(size[-1], 1e-6)
        caches.append((f"L{level}", float(size.rstrip("KM")) * factor / 1e6))

print(f"{'numbers in each array':>22s}{'MB of the three arrays':>24s}{'additions per second':>22s}")
for length, mb, rate in zip(lengths, megabytes, sweep):
    print(f"{length:22,d}{mb:24.3f}{in_words(rate):>22s}")
print("Caches of this processor:", ", ".join(f"{name} {mb:g} MB" for name, mb in caches) if caches
      else "the system of this machine does not tell them.")

fig, ax = plt.subplots(figsize=(9, 4.6))
ax.plot(megabytes, [rate / 1e9 for rate in sweep], marker="o", color="#2a78d6")
for name, mb in caches:
    ax.axvline(mb, color="#888888", linestyle=":", linewidth=1)
    ax.annotate(f"{name}: {mb:g} MB", (mb, max(sweep) / 1e9), xytext=(4, -4), textcoords="offset points",
fontsize=9, va="top")
ax.set_xscale("log")
ax.set_xlabel("MB of the three arrays a, b and c (each mark on the axis is 10 times the one before)")
ax.set_ylabel("billions of additions a second")
ax.set_ylim(0, max(sweep) / 1e9 * 1.15)
ax.set_title("The rate of a + b, against the size of the arrays")
ax.grid(True, which="major", color="#e1e0d9", linewidth=0.8)
ax.spines[["top", "right"]].set_visible(False)
fig.tight_layout()
plt.show()

print("Line for the discussion board:")
print(f"a + b: {in_words(max(sweep))} additions a second at best, at
{megabytes[sweep.index(max(sweep))]:.2g} MB; "
      f"{in_words(sweep[-1])} at {megabytes[-1]:.0f} MB.")
```

OUTPUT

```
 numbers in each array  MB of the three arrays  additions per second
                 1,024                   0.012          2.46 billion
                 2,048                   0.025          4.16 billion
                 4,096                   0.049          5.16 billion
                 8,192                   0.098          6.94 billion
                16,384                   0.197          8.97 billion
                32,768                   0.393          9.23 billion
                65,536                   0.786          11.3 billion
               131,072                   1.573          8.73 billion
               262,144                   3.146          4.53 billion
               524,288                   6.291          4.38 billion
             1,048,576                  12.583          4.39 billion
             2,097,152                  25.166          4.24 billion
             4,194,304                  50.332          3.08 billion
             8,388,608                 100.663          2.36 billion
            16,777,216                 201.327          2.26 billion
            33,554,432                 402.653          2.19 billion
Caches of this processor: the system of this machine does not tell them.
Line for the discussion board:
a + b: 11.3 billion additions a second at best, at 0.79 MB; 2.19 billion at 403 MB.
```

1.6 Predict the time of a layer, then measure it Part 6

The count and the rates together predict the time of a computation before it is run: **time ≈ operations / operations per second**. A layer of K neurons on n points with m coordinates takes $2nKm$ operations; a layer of 1,000 neurons on 60,000 small grey images of 28 by 28 pixels, $m = 784$, takes about 94 billion, and the rates of Part 4 and of the class's posts predict for it 2.33 hours with the three loops of Python, 1.11 seconds on Colab's CPU, and 33.3 milliseconds on its T4. Each group chose a layer of its own, below a thousand billion operations and two gigabytes of numbers, and the notebook predicted its time with the loops, on the CPU and on the GPU, then measured it on the two processors; the loops are not run, since they would take hours.

The layer of these notes is 50,000 colour images of 32 by 32 pixels, $m = 3,072$, through 512 neurons: 157 billion operations. The CPU of this machine was predicted to take 214 milliseconds and took 224 milliseconds, a ratio of 1.05. A prediction and a measurement differ for reasons the notebook has already met: the rate was measured on a square product and this layer is not square; the machine was doing other things; the first use of a size costs extra. Arithmetic intensity also matters: a layer with very few neurons, $K = 4$, does only two operations for each byte of `X` it reads, and is memory-bound, so it runs far below the rate of a large product, on the CPU and on the GPU alike. That is the question of the last thinking cell, and section 3.2 answers it on the roofline.

Code: Part 6, the layer of these notes, the prediction and the measurement, and the predictions for the layer of 94 billion operations

cells `choose-layer`, `layer-predict`, `recap-predict`

```
n_layer, m_layer, K_layer = 50_000, 3_072, 512      # 50,000 colour images of 32 by 32 pixels, a layer of
```

512 neurons

OUTPUT

```
A layer of K = 512 neurons on n = 50,000 points with m = 3,072 coordinates: 157 billion
operations, 0.72 GB of numbers.
way of computing                 predicted            measured   measured / predicted
Python: three loops             3.89 hours             not run
CPU                        214 milliseconds    224 milliseconds                    1.05

Line for the discussion board:
Layer n = 50,000, m = 3,072, K = 512: 157 billion operations. Python: three loops: predicted 3.89
hours; CPU: predicted 214 milliseconds, measured 224 milliseconds.
```

OUTPUT

```
K = 1,000 neurons, n = 60,000 images of 28 x 28 pixels (m = 784): 94.1 billion operations
implementation                         predicted time
Python: three loops                        2.33 hours
CPU, 32 logical processors: X @ W.T + b  128 milliseconds
Colab CPU, 2 cores: X @ W.T + b         1.11 seconds
Colab T4: X @ W.T + b              33.3 milliseconds
```

The two posts.

The first notebook asks for two posts on the discussion board of the meeting, before the meeting begins, each beginning with the names of all the members of the group: after Part 2, the line printed under the chart, the chart with one sentence that says what it shows, and the two factors; after Part 6, as a reply, the rates of Part 4 and the sweep of Part 5 with their lines and charts, and the line of Part 6. A post earns the attention mark of the meeting for everyone it names. The second notebook posts nothing.

In the meeting.

Recap 1 went over this notebook from the class's own numbers. The dot plot of idea 3 is the twelve T4 posts of meeting 11; the board of meeting 12 had 58 posts by the meeting, and the ones read that morning agreed with them: the T4 at 0.0486 to 0.0487 seconds for the product at size 4,096 in every session that had one, Colab's two cores between 1.0 and 2.0 seconds, so the GPU 21 to 41 times as fast, and one student on a desktop with an AMD GPU at 7.6 trillion operations a second against 670 billion for its 16-core CPU. The rates of Part 4 came out as the chart of idea 2 shows: about 5 million operations a second for the three loops of Python, about a billion for `a + b` on long arrays and 1.5 to 4.7 billion on short ones, 80 to 130 billion for the product on Colab's CPU, 2.8 trillion on the T4. The sweeps of Part 5 peaked at 3 to 5 billion additions a second near 0.8 MB and fell to about a billion at 403 MB: Colab's caches are smaller than the ones marked in these notes. The predictions of Part 6 came within a factor of two of the measurements on both processors, as the recap expected; one group's layer of 583 billion operations was predicted at 4.6 seconds on the CPU and took 6.7, and at 206 milliseconds on the T4 and took 160. The recap's last slide, *What O( ) leaves out*, turned the meeting to the second notebook.

The second notebook. What Big O leaves out

Open the second notebook (https://colab.research.google.com/github/ikoutis/cs301-labs/blob/main/unit-2/notebooks/meeting-12-rows-and-columns.ipynb)

2.1 A matrix is one line of numbers Part 1

Every byte of the memory has an **address**, and the addresses run in one sequence. The contiguous row-major NumPy array used here stores its entries in consecutive memory locations. For a matrix with 3 rows and 4 columns of 8-byte numbers, `X.ravel()` shows the line as it is stored, the rows one after another, **row order**, and `X.strides` gives `(32, 8)`: 32 bytes to go down one row, past the four numbers of a row, 8 bytes to go right by one column. So the number in row i and column j of a matrix with m columns is at position $i \times m + j$ of the line, after i whole rows and j numbers of its own row, and the part asks for that function, `position(i, j, m)`, checked on every entry of a matrix that is not square. The check recognizes column-major indexing, $j \times n + i$, which is the position in a line that stores the columns one after another.

The transpose is the same line read in another way: `X.T` makes no copy and moves no number, it swaps the two strides, so that one step along a row of `X.T` is a step down a column of `X`, and `np.shares_memory` confirms that the two arrays are one block. Other languages, Fortran, MATLAB and R among them, store a matrix in **column order**, one column after another; `np.asfortranarray` makes such a copy, with the strides `(8, 24)`.

Code: Part 1, the line, the function position, its check, and the transpose — cells `one-line`, `paste-position`, `part1-check`, `strides`

```
X = np.arange(12.0).reshape(3, 4)        # the numbers 0 to 11, as 3 rows of 4
print(X)
print("the line in memory:", X.ravel())
print("bytes for one number:", X.itemsize, "  strides:", X.strides)
```

OUTPUT

```
[[ 0.  1.  2.  3.]
 [ 4.  5.  6.  7.]
 [ 8.  9. 10. 11.]]
the line in memory: [ 0.  1.  2.  3.  4.  5.  6.  7.  8.  9. 10. 11.]
bytes for one number: 8   strides: (32, 8)
```

```
def position(i, j, m):
    """The position, in the line of memory, of the number in row i and column j of a matrix with m columns."""
    return i * m + j
```

OUTPUT

```
Correct: every number of a matrix with 5 rows and 7 columns is where position says.
```

```
XT = X.T
print("X.T has shape", XT.shape, "and strides", XT.strides, "  same memory as X:", np.shares_memory(X, XT))
C = np.asfortranarray(X)                 # the same table, stored one column after another
print("in column order, the line is:", C.ravel(order="K"), "  strides:", C.strides)
```

OUTPUT

```
X.T has shape (4, 3) and strides (8, 32)   same memory as X: True
in column order, the line is: [ 0.  4.  8.  1.  5.  9.  2.  6. 10.  3.  7. 11.]   strides: (8, 24)
```

2.2 Loops in Python, by row and by column Part 2

Two functions add up every number of a matrix with loops in Python alone. The first goes along the rows, in the order of the line; the second goes down the columns, and each of its steps jumps a whole row ahead in the line. On a matrix of 2,000 by 2,000 numbers, 32 megabytes, this machine took 91 nanoseconds for each number by rows and 93 by columns, a ratio of 1.02; in another run of the same cell on the same machine the ratio was 1.05, and Part 3 finds a far larger one for the same two orders once the loop over the numbers is NumPy's. The question of the part is why the order shows so little in loops written in Python. Each pass of a Python loop costs tens of nanoseconds of work by the interpreter: it finds the objects that the names refer to, builds the pair of indices, calls NumPy to fetch one number from the block, and makes a Python object for it. This overhead can mask the

difference in memory-access cost. The next part reduces it by using NumPy to process a whole row or column at once.

Code: Part 2, the two loops, timed — cell `python-loops`

```
def sum_by_rows(X):
    """Every number of X, row after row: the order of the line."""
    n, m = X.shape
    total = 0.0
    for i in range(n):
        for j in range(m):
            total += X[i, j]
    return total


def sum_by_columns(X):
    """Every number of X, column after column: a jump of a whole row at each step."""
    n, m = X.shape
    total = 0.0
    for j in range(m):
        for i in range(n):
            total += X[i, j]
    return total


n = 2000
X = rng.random((n, n))
t_rows = best_time(lambda: sum_by_rows(X), repeats=2)
t_columns = best_time(lambda: sum_by_columns(X), repeats=2)
print(f"{n} x {n} numbers, {X.nbytes / 1e6:.0f} MB")
print(f"by rows:    {t_rows:.3f} s, {t_rows / n ** 2 * 1e9:.0f} nanoseconds for each number")
print(f"by columns: {t_columns:.3f} s, {t_columns / n ** 2 * 1e9:.0f} nanoseconds for each number")
print(f"by columns / by rows: {t_columns / t_rows:.2f}")
```

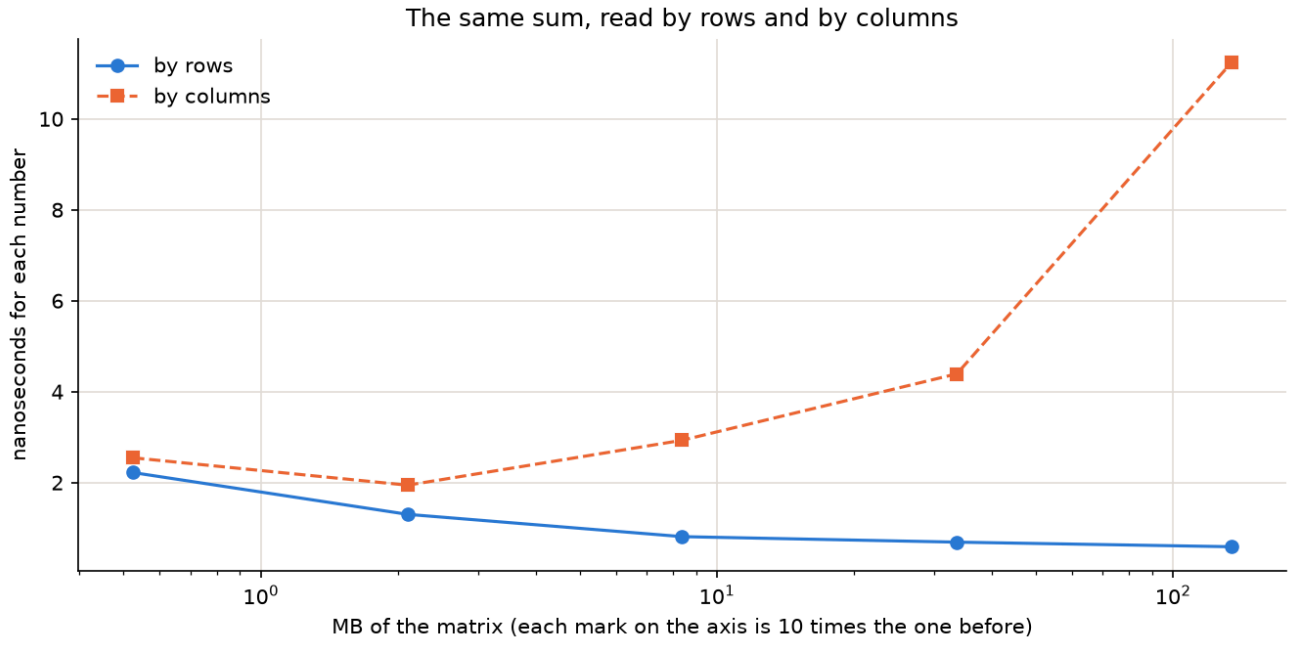
OUTPUT

```
2000 x 2000 numbers, 32 MB
by rows:    0.365 s, 91 nanoseconds for each number
by columns: 0.372 s, 93 nanoseconds for each number
by columns / by rows: 1.02
```

2.3 Rows and columns, with NumPy inside the loop Part 3

The loop over the numbers was the expensive part, so this part leaves one loop in Python and gives the rest to NumPy: `sum_by_rows_numpy` loops over the rows and hands each row, `X[i, :]`, a block of m numbers side by side in the line, to NumPy to add up. The part asks for the same function with a loop over the columns, each column `X[:, j]` being the numbers at the same position of every row. The check reads the syntax of the function rather than its text: it needs a loop, and a column or a transpose inside it. The two functions are then timed on square matrices of five sizes, from 256 by 256 numbers, half a megabyte, to 4,096 by 4,096, 134 megabytes.

The result on this machine: the two orders have similar measured costs for the matrices of half a megabyte and of two megabytes, which fit in the L2 cache of a core; beyond it the columns cost more, 3.6 times as much at 8 megabytes, 6.4 times at 34, and 19.1 times at 134, beyond the L3 cache of 36 megabytes that the cores share. (The smallest matrices cost several nanoseconds for each number in both orders: a row of 256 numbers is a short call to NumPy, and the fixed cost of a call shows, as it did at the far left of the sweep of Part 5.) The reason for the difference is the cache line of idea 5: memory transfers to the cache use 64-byte lines, eight numbers of this matrix that sit side by side; a row uses all eight of every line it fetches, and sequential access allows prefetching; a column uses one number of each line and jumps a whole row ahead. While the matrix fits in the cache the other seven numbers are still there when the next column needs them; beyond the cache they may be evicted before reuse, requiring another read from main memory. Where the order begins to matter is therefore where the matrix outgrows the cache nearest the core: on this machine the ratio climbs from the first size beyond L2 and is largest beyond L3. In Colab the dotted lines of the chart mark the sizes of its processor's caches.



The same sum, read by rows and by columns, at five sizes, on the machine that built these notes.

Code: Part 3, the two functions, the check, and the measurement

cells `paste-columns`, `part3-check`, `rows-columns`

```
def sum_by_rows_numpy(X):
    """Every number of X: a loop over the rows, and NumPy adds up each row."""
    total = 0.0
    for i in range(X.shape[0]):
        total += X[i, :].sum()
    return total
```

```python
def sum_by_columns_numpy(X):
    """Every number of X: a loop over the columns, and NumPy adds up each column."""
    total = 0.0
    for j in range(X.shape[1]):
        total += X[:, j].sum()
    return total
```

OUTPUT

```
Correct: sum_by_columns_numpy adds up every number, one column at a time.
```

```python
sizes = [256, 512, 1024, 2048, 4096]
by_rows, by_columns, megabytes = [], [], []
for n in sizes:
    X = rng.random((n, n))
    megabytes.append(X.nbytes / 1e6)
    by_rows.append(best_time(lambda: sum_by_rows_numpy(X)) / n ** 2 * 1e9)
    by_columns.append(best_time(lambda: sum_by_columns_numpy(X)) / n ** 2 * 1e9)
del X

print(f"{'size':>6s}{'MB':>8s}{'by rows, ns a number':>23s}{'by columns':>13s}{'columns / rows':>17s}")
for n, mb, r, c in zip(sizes, megabytes, by_rows, by_columns):
    print(f"{n:6d}{mb:8.1f}{r:23.2f}{c:13.2f}{c / r:17.1f}")
```

OUTPUT

```
  size      MB   by rows, ns a number   by columns   columns / rows
   256     0.5                   2.22         2.55              1.1
   512     2.1                   1.30         1.94              1.5
  1024     8.4                   0.81         2.93              3.6
  2048    33.6                   0.69         4.39              6.4
  4096   134.2                   0.59        11.24             19.1
```

2.4 A power of two, and one more Part 4

Sizes that are powers of two are everywhere: images, layers and batches are often 1,024, 2,048 or 4,096 wide. The part reads matrices by columns, with the function of Part 3, at these three sizes and at their neighbours, one number smaller and one larger, and asks for a prediction first: a matrix with 2,049 columns has more numbers than one with 2,048; should each of its numbers cost more to read? On this machine the matrix of 2,048 columns cost 7.01 nanoseconds for each number, its neighbours 2.67 and 2.38; the chart is in idea 6.

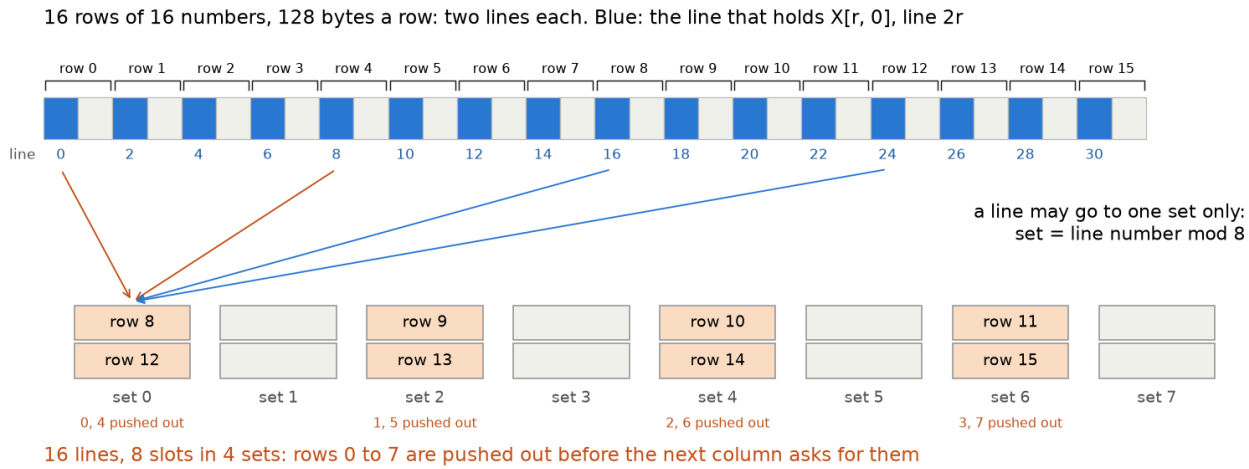
A cache is divided into **sets**; the address of a line decides which set it goes to, and each set holds a limited number of lines. In a simple modulo-indexed model, consecutive lines go to consecutive sets. A large power-of-two row stride can instead send a column's lines to a small fraction of the sets. Those sets can fill even when others have unused space, causing lines to be evicted before the next column needs them. Padding changes the stride and can reduce these conflicts.

The drawings use this model with 64-byte lines, eight `float64` numbers per line, a base address of zero, an initially empty cache, and least-recently-used (LRU) replacement. They show the state after reading column 0 in row order, with no prefetching or other accesses. For 16 rows of 16

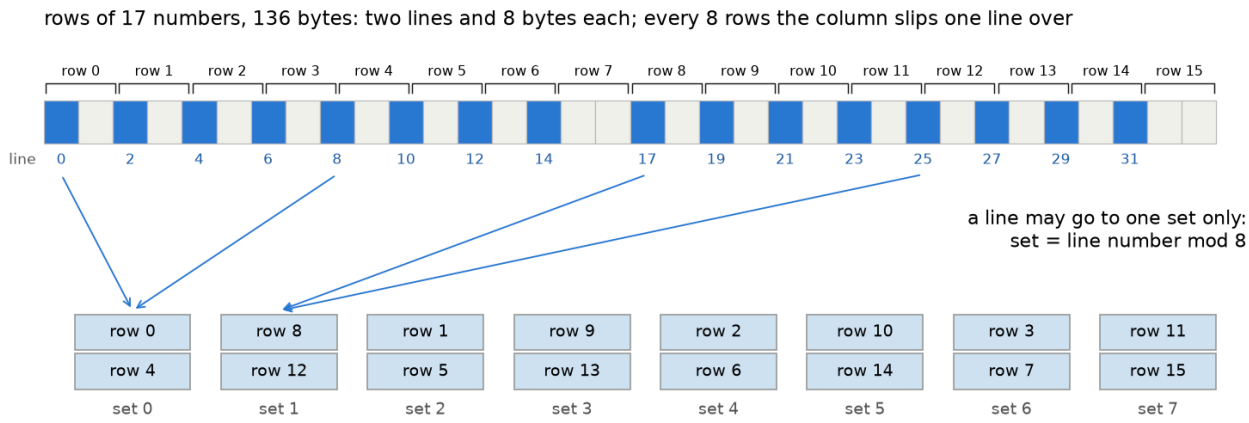
numbers, the first number of row r lies in line $2r$. With 8 sets of 2 lines, its set is $2r \bmod 8$: only sets 0, 2, 4 and 6 are used. Sixteen lines compete for eight slots, and rows 0 to 7 have been evicted by the end of column 0. For rows of 17 numbers, the first-column line is $2r + \lfloor r / 8 \rfloor$. Rows 0 to 7 use the even sets twice each, and rows 8 to 15 use the odd sets twice each. All sixteen lines fit at the end of column 0. This does not guarantee a hit on every read of column 1: rows 7 and 15 cross into new lines. With least-recently-used (LRU) replacement, row 7 also evicts the line needed by row 11, giving three misses in column 1.

For a larger example, use the capacities of this laptop's performance core: the L1 data cache is 48 KiB with 12 lines per set, so it has $48 \cdot 1024 / (64 \cdot 12) = 64$ sets; L2 is 2 MiB with 16 lines per set, so it has $2 \cdot 1024^2 / (64 \cdot 16) = 2048$ sets. In the same aligned model, a row of 2,048 numbers is 256 lines, and $256r \bmod 2048$ takes eight values. For 2,048 rows, 256 lines compete for each set's 16 slots, 128 slots in total. With 2,049 numbers per row, the first-column line is $256r + \lfloor r / 8 \rfloor$. Write $r = 8q + t$, where $0 \le q < 256$ and $0 \le t < 8$. Its set is $256t + q$, which visits each of the 2,048 sets once.

That exact count applies to column 0 and a line-aligned base in this model. The actual L2 placement depends on physical address mapping, which a contiguous NumPy array does not establish. Alignment, prefetching, replacement, and other cache levels also affect the measured result. The model explains a source of conflicts; its set counts alone do not predict a timing ratio.



Rows of 16 numbers, two lines each: the line that holds the first number of row r is line 2r, and it may go only to set 2r mod 8. Sixteen lines, eight slots.



Rows of 17 numbers, two lines and 8 bytes each: every 8 rows the column slips one line over, into an odd set. After column 0, all sixteen fetched lines fit, two per set.

The fix is one more number in each row, **padding**: a matrix of 2,048 columns stored in rows of 2,049 numbers, the last of each row unused. In NumPy, `np.zeros((n, m + 1))[:, :m]` is such an array: its shape is (n, m) and its rows are $m + 1$ numbers apart in the line. The part asks for the function `padded(X)` that makes this copy, and the check times the columns of a matrix of 2,048 by 2,048 stored as it is and padded: on this machine 26.6 milliseconds against 11.5, a factor of 2.3. Matrix libraries can apply padding internally to reduce these conflicts.

Code: Part 4, the measurement around three powers of two, the function padded, and its check

cells `pow2`, `paste-padded`, `part-pow2-check`

```
groups = [1024, 2048, 4096]
sizes_pow2 = [n for k in groups for n in (k - 1, k, k + 1)]
pow2_ns = {n: float("inf") for n in sizes_pow2}
for k in groups:
    matrices = {n: rng.random((n, n)) for n in (k - 1, k, k + 1)}
    for _ in range(5):                    # five rounds over the three sizes: the best time of each is kept
        for n, X in matrices.items():
            start = time.perf_counter()
            sum_by_columns_numpy(X)
            pow2_ns[n] = min(pow2_ns[n], (time.perf_counter() - start) / n ** 2 * 1e9)
del matrices, X

print(f"{'size':>6}{'by columns, ns a number':>26s}")
for n in sizes_pow2:
```

OUTPUT

```
  size   by columns, ns a number
  1023                      1.04
  1024                      2.52   <- a power of two
  1025                      1.09
  2047                      2.67
  2048                      7.01   <- a power of two
  2049                      2.38
  4095                      5.37
  4096                     11.47   <- a power of two
  4097                      5.05
```

```
def padded(X):
    """A copy of X stored in rows one number longer than X's: the same table, with one unused number after
each row."""
    n, m = X.shape
    P = np.zeros((n, m + 1))
    P = P[:, :m]
    P[:] = X
    return P
```

OUTPUT

```
Correct: padded keeps the table and stores each row one number longer.

The columns of a matrix of 2,048 by 2,048 numbers:
stored as it is, rows of 2,048 numbers:     26.6 milliseconds
padded, rows of 2,049 numbers:            11.5 milliseconds
as it is / padded: 2.3
```

2.5 Column sums, read by rows Part 5

Often the result itself is by columns: the mean of every column of a table, for instance, needs the sum of each column. A straightforward loop goes down each column, which Part 3 found slow. The same sums can be collected while the matrix is read by rows: keep m running sums, one for each column, and add each whole row to them. The part asks an AI assistant for this function, `column_sums_by_rows(X)`, to four requirements: its name and argument; an array of m sums returned; one loop, over the rows, adding each whole row to an array of zeros; and no loop over the columns, no column `X[:, j]`, and no call of `X.sum` or `np.sum`. The check examines executable statements; mentioning `X.sum(axis=0)` in a docstring is allowed. On a matrix of 134 megabytes, this machine took 191.4 milliseconds for a loop over the columns, 6.7 for the loop over the rows, and 4.3 for NumPy's own `X.sum(axis=0)`, which does the same thing by itself.

Code: Part 5, a function written to the requirements, and its check with the timing — cells `paste-column-sums`, `part4-check`

```
def column_sums_by_rows(X):
    """The sum of each column of X, read row by row: X.sum(axis=0) without NumPy's sum."""
    total = np.zeros(X.shape[1])
```

```
    for i in range(X.shape[0]):
        total += X[i]
    return total
```

OUTPUT

```
Correct: column_sums_by_rows returns the sum of every column, reading the matrix by rows.

The sums of the 4,096 columns of a matrix of 134 MB:
a loop over the columns, X[:, j].sum():      191.4 milliseconds
column_sums_by_rows, a loop over the rows:      6.7 milliseconds
NumPy's own X.sum(axis=0):                      4.3 milliseconds
```

2.6 What NumPy does by itself Part 6

NumPy can choose an efficient memory traversal independently of which output axis is requested. The last part times four pairs of operations on a matrix of 4,096 by 4,096 numbers; in each pair the second reads the same numbers in another order. A sum down the columns, `X.sum(axis=0)`, can avoid the penalty of the explicit column loop. Its measured ratio to a sum along the rows is 0.6 here. NumPy can collect column sums while reading contiguous rows, as Part 5 did. A product with the matrix stored in column order, `F @ v`, has a measured ratio to row order of 1.4 times. An operation that must read one matrix by rows and the same matrix, transposed, by columns cannot choose: `X.T.copy()` costs 10.0 times as much as `X.copy()`, and `X + X.T` 8.3 times as much as `X + X`, although each pair adds or copies the same number of pairs of numbers. These pairs show where memory layout affects performance even in optimized NumPy operations.

Code: Part 6, four pairs of operations — cell `numpy-pairs`

```
n = 4096
X = rng.random((n, n))
F = np.asfortranarray(X)                # the same matrix, stored in column order
v = rng.random(n)
pairs = [("X.sum(axis=1)", lambda: X.sum(axis=1), "X.sum(axis=0)", lambda: X.sum(axis=0)),
         ("X @ v", lambda: X @ v, "F @ v, F in column order", lambda: F @ v),
         ("X.copy()", lambda: X.copy(), "X.T.copy()", lambda: X.T.copy()),
         ("X + X", lambda: X + X, "X + X.T", lambda: X + X.T)]
print(f"{'operation':16s}{'ms':>8s}    {'the same numbers, another order':32s}{'ms':>8s}{'ratio':>8s}")
for name_a, a, name_b, b in pairs:
    ta, tb = best_time(a), best_time(b)
    print(f"{name_a:16s}{ta * 1e3:8.1f}    {name_b:32s}{tb * 1e3:8.1f}{tb / ta:8.1f}")
```

OUTPUT

```
operation             ms    the same numbers, another order       ms   ratio
X.sum(axis=1)        7.3    X.sum(axis=0)                         4.4     0.6
X @ v                3.0    F @ v, F in column order              4.3     1.4
X.copy()            20.6    X.T.copy()                          205.5    10.0
X + X               24.7    X + X.T                             205.1     8.3
```

Image layouts and strides.

An image of h rows and w columns of pixels with three colours has shape $(h, w, 3)$ and is stored pixel after pixel: the red, green and blue numbers of one pixel sit side by side, and the red channel `image[:, :, 0]` is every third number of the line. PyTorch image layers commonly use the logical shape $(N, 3, h, w)$. A contiguous array in that order stores each channel as a block, but strides determine the physical layout. Permuting dimensions can create a view without moving data.

Memory: the opening and Deeper 1

This section brings together the memory topics from the opening and Deeper 1: the cache hierarchy and measured bandwidth, then operations per byte, the roofline, blocking, and GPU transfers. In class, the two recaps come between these parts. Its figures of the T4 come from NVIDIA's data sheet; its measurements were made on the machine that built these notes, with the cells of the first notebook.

3.1 Bytes a second at each level

The sweep of Part 5, run again for these notes with the best of five runs kept, samples three size ranges: the three arrays fitting in L2, fitting in L3, and exceeding both caches. Each addition reads two numbers of 4 bytes and writes one, 12 bytes in all, so the rate of additions in each range estimates the **effective bandwidth** of that level, the bytes it delivers in a second to one core, since `np.add` uses one core. The table uses the median rate within each size range. Its byte counts describe useful array traffic; hardware transfers can also include write allocation and other overhead.

OUTPUT

the three arrays	capacity	additions a second	bytes a second
in L2	2 MB, one core	9.81 billion	118 billion
in L3	36 MB, shared	4.94 billion	59.3 billion
in main memory	gigabytes	2.28 billion	27.4 billion

The drawing of idea 7 puts these numbers over the hierarchy: a core, its L1 of 48 kilobytes and its L2 of 2 megabytes, the L3 of 36 megabytes shared by all the cores, the main memory of gigabytes, and, across the link, the GPU's memory, 16 gigabytes at 300 gigabytes a second by the T4's data sheet. The link is PCI Express, 32 gigabytes a second in both directions together, 16 each way. The CPU's memory hierarchy trades access speed for capacity. `a + b` is memory-bound at every level: it does too little arithmetic for each byte to keep the core busy, and its rate is the bandwidth divided by 12.

3.2 Operations per byte, and the roofline

What decides whether a computation is memory-bound or compute-bound is its **operations per byte**: the arithmetic it does divided by the bytes it moves to and from memory.

$$a + b\text{: } \frac{1 \text{ operation}}{12 \text{ bytes}}, \qquad XW^T \text{ at size } N\text{: } \frac{2N^3 \text{ operations}}{3 \times 4N^2 \text{ bytes}} = \frac{N}{6}.$$

Multiplying two matrices of size N takes $2N^3$ operations; reading both and writing the result once moves $12N^2$ bytes; so the product does $N / 6$ operations for each byte, 683 at $N = 4,096$, under the idealized assumption that each number is fetched once and reused N times. Section 3.4 explains how blocking reduces repeated memory reads. The **roofline** draws both limits of a processor on one chart, with operations per byte along the bottom and operations a second up the side: a sloping line, the bandwidth times the operations per byte, and a flat line, the peak rate of the arithmetic. A computation can go no faster than the lower of the two,

$$\text{speed} \le \min\left(\text{peak rate}, \text{bandwidth} \times \text{operations per byte}\right),$$

and the two lines meet at the **ridge**: for the T4, 8.1 trillion operations a second divided by 300 gigabytes a second, 27 operations per byte. Left of the ridge the memory sets the speed; right of it the arithmetic does. The chart of idea 8 places three computations under the T4's roof. `a + b`, at one operation for 12 bytes, can reach at most 25 billion additions a second on the T4, under this bandwidth limit. A layer with $K = 4$ neurons does about two operations for each byte of `X` it reads and is memory-bound even on a GPU, which is why a prediction from the rate of a large product would be far too optimistic for it, the question of Part 6. The product at size 4,096 is far to the right, under the flat part of the roof, and the class's measurement of it, 2.8 trillion operations a second, sits below the peak.

3.3 The T4 reached a third of its peak

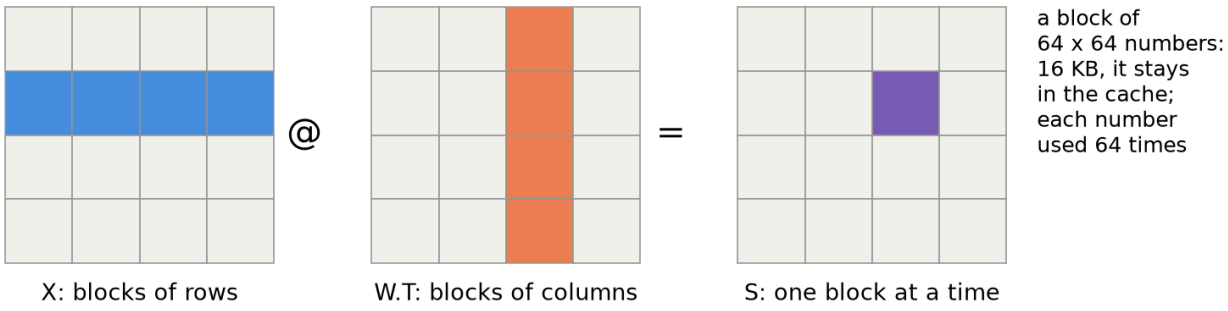
The T4	Operations a second	The product at size 4,096
its data sheet: the peak	8.1 trillion	0.017 s
published benchmarks of cuBLAS on Colab	4.6 trillion	0.030 s
the class's posts, the median of twelve	2.8 trillion	0.049 s

The product is compute-bound, and the class still measured about a third of the peak of the data sheet. The peak is an arithmetic upper bound, every multiplier busy every cycle, rather than a guaranteed application rate. Published benchmarks of cuBLAS, the library that computes the product on NVIDIA's GPUs, reach about 4.6 trillion operations a second on a Colab T4 (github.com/gabmerlo/cuda-kernels; github.com/aaron-ang/cuda-sgemm). Why the class's numbers

are below the benchmarks is not established. Two candidates: the benchmarks repeat the product many times and report the best or the median run, while the notebook times one product at that size; and the T4 lowers its clock under its limit of 70 watts, so that a product that follows other work may run slower than one that starts cold.

3.4 Computing the product in blocks

The count of a product does not say in which order its operations are done, and the order decides how many bytes move. A library does not compute `X @ W.T` row by row from the main memory. It divides `X` into blocks of rows and `W.T` into blocks of columns, small enough for a block of each to sit together in the cache, 64 by 64 numbers, 16 kilobytes each, for instance, and computes one block of the result from them, using every number of the two blocks 64 times while they are there. The count remains $2nKm$, but reusing cached blocks reduces memory traffic and raises the operations per byte discussed in section 3.2. This is why the product of Part 4 did not slow down when its matrices outgrew the cache, while `a + b` did.



The product in blocks: one block of rows of X and one block of columns of W.T make one block of S, and both stay in the cache while they are used.

3.5 When copying takes longer than computing

The GPU computes only with numbers in its own memory, and they cross the link to get there, at 16 gigabytes a second each way by the T4's data sheet. The table divides the bytes of two computations by that bandwidth, a lower bound on the time of the copies, and sets it beside the time of the work: the class's own for the product, and the bandwidth of the T4's memory for the addition, since `a + b` can go no faster than its memory.

OUTPUT

computation	copies across the link	GPU compute time
X @ W.T at size 4,096 on the T4	12.6 milliseconds	48.7 milliseconds
a + b on 16.8 million numbers on the T4	12.6 milliseconds	0.671 milliseconds

For the product at size 4,096, the three matrices are 201 megabytes and cross the link in at least 12.6 milliseconds, against 48.7 milliseconds of work: the copies are about a quarter of the work. For `a + b` on 16.8 million numbers, the same 201 megabytes cross the link in the same time, and the ideal bandwidth-based addition time is under a millisecond. The estimated copy time is about twenty times that lower bound. A program that uses a GPU keeps its numbers there across many operations and copies as little as it can; that is the answer to the question of Part 2 about the copy of one matrix. In a Colab session with a T4, the addition itself can be measured with the arrays already on the GPU, and the roofline predicts its result before it is run: at most 300 gigabytes a second, 25 billion additions a second.

```
n = 2 ** 26                                          # 67 million numbers in each array
a = torch.rand(n, device="cuda")
b = torch.rand(n, device="cuda")
c = torch.empty(n, device="cuda")
torch.add(a, b, out=c); torch.cuda.synchronize()     # once, not measured
start = time.perf_counter()
for _ in range(20):
    torch.add(a, b, out=c)
torch.cuda.synchronize()
seconds = (time.perf_counter() - start) / 20
print(f"{n / seconds / 1e9:.1f} billion additions a second, {12 * n / seconds /
1e9:.0f} GB/s")
```

A simple running-time estimate uses the larger of the arithmetic and memory-transfer times:

$$\text{time} \approx \max\left(\frac{\text{operations}}{\text{peak rate}}, \frac{\text{bytes moved}}{\text{bandwidth}}\right).$$

O( ) bounds the growth of the arithmetic count. This roofline expression is an ideal lower-bound estimate using peak rates; actual running time can be longer. The operations per byte select the limiting term, and where the numbers are, the cache, the main memory or the GPU's memory, sets the bandwidth.

In the meeting.

The deeper look closed with the code above, for a Colab session with a T4, and the roofline's prediction for it: at most 300 gigabytes a second, 25 billion additions a second, whatever the arithmetic peak. No number from the meeting is recorded here; run the cell in a session with a T4 and compare your own.

A deeper look 2. What computing costs

4.1 Three costs of one computation

$$\text{time} = \frac{\text{operations}}{\text{operations per second}}, \qquad \text{energy} = \text{power} \times \text{time}, \qquad \text{money} = \text{price per hour} \times \text{hours}.$$

The first cost is the count and the rate of this meeting; the other two follow from it. Power is measured in **watts**, joules each second, and electricity is sold by the **kilowatt-hour**, a thousand watts for an hour, 3.6 million joules. The power that a processor may draw is on its data sheet, and it is an upper bound: a processor may draw less.

Processor	Power	Source
Colab's T4	70 W at most	NVIDIA's data sheet
the CPU of the machine of these notes, an Intel Core i9-14900HX	55 W at its base speed, 157 W at most	Intel
a V100 training GPT-3	330 W, with its share of the server	measured; Patterson et al., 2021

Colab does not publish the model or the power of its CPUs.

4.2 The joules of one product

OUTPUT

```
X @ W.T at size 4,096: 137 billion operations
processor                              seconds  watts   joules   operations a joule
CPU of this machine, 32 logical cores    0.187    157     29.3         4.69 billion
Colab's T4: your posts                   0.049     70      3.4         40.3 billion
```

The CPU row is this machine's own time for the product of Part 2, at the 157 watts it may draw at most; the T4 row is the class's median time at the T4's 70 watts. Both are upper bounds on the energy. At these rated-power estimates, the GPU has more operations per joule: 40.3 billion against 4.69 billion. These estimates do not establish the ratio of actual energy consumption. That would require measuring power or energy during the runs. At data-centre scale, both processor time and energy contribute to cost.

4.3 Dollars an hour, and the dollars of one product

	Price	Source
a T4, rented by the hour	$0.35	Google Cloud, on demand, Iowa region, the GPU alone
a V100, rented by the hour	$2.48	Google Cloud, on demand, the same
Colab's T4	free, when one is spare	Google Colab
electricity, a kilowatt-hour	15 cents	U.S. average over all sectors, July 2026, EIA

The cloud prices are for the GPU alone; the virtual machine's processor, memory and disk cost extra. An hour of a T4 at full power uses 0.07 kilowatt-hours, about one cent of electricity, so the rent is about thirty times the electricity: the rental price also covers hardware, infrastructure, cooling, and service costs. On a rented T4, in cents:

OUTPUT

```
on a rented T4, at $0.35 an hour, in cents              rent     electricity
one hour                                                  35               1
one product at size 4,096                            0.00047        0.000014
a trillion operations, at your rate                   0.0034          0.0001
the layer of the notebook: 94 billion                 0.00032       0.0000097
```

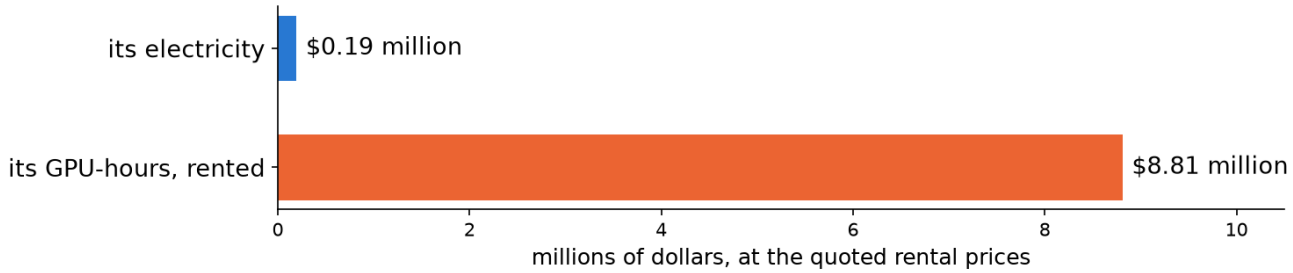
A second of a T4 costs about a hundredth of a cent, and one product at size 4,096 takes 0.049 seconds of it. At the class's rate, a trillion operations cost about 0.0034 cents of rent. Small per-operation costs accumulate over a large training run.

4.4 A model trained at scale: GPT-3

GPT-3, the language model of 2020	
operations to train it	3.14×10^{23}
processors	10,000 V100 GPUs
time	14.8 days
power of each, with its share of the server	330 W
energy	1,287 MWh
carbon dioxide, and its equivalents	552 tonnes

OpenAI published the count of operations of the training of GPT-3, the model with 175 billion weights; Patterson and colleagues (*Carbon Emissions and Large Neural Network Training*, 2021) estimated the rest from figures that OpenAI and NVIDIA gave: ten thousand V100 GPUs for about two weeks, 1,287 megawatt-hours, and 552 tonnes of carbon dioxide and equivalent gases, about three round trips of a passenger jet between San Francisco and New York by the paper's own comparison. Using the rates and prices above:

```
OUTPUT
GPT-3: 3.14 x 10^23 operations
on one T4 at your rate:              3,526 years
on 10,000 V100s, as it was trained:   14.8 days, 3.55 million GPU-hours
its electricity, 1,287 MWh, at today's U.S. average price:  $192,921
its GPU-hours at the quoted list price for a V100:          $8,808,960
```



The two bills of GPT-3's training at today's prices: the electricity, and the GPU-hours rented at Google Cloud's list price for a V100.

At the class's own rate, 2.8 trillion operations a second, one T4 would need about 3,526 years; ten thousand V100s did it in two weeks. The electricity of the run, at the quoted U.S. electricity price, would be about $192,921; the same GPU-hours rented at the quoted prices at Google Cloud's list price for a V100, about $8,808,960. Most of the bill is the rent of the processors, not the electricity. What OpenAI paid in 2020 is not public; the second number is an illustration at the quoted list price.

4.5 What decides the cost

$$\text{cost} \approx \text{count} \times \text{seconds for one operation} \times \text{price of a second}.$$

The operation count is independent of the machine. O( ) describes its growth with input size. Time per operation depends on the implementation, access order, cache, and processor. Power and hourly rental price convert elapsed time into energy and monetary estimates. Compare achieved throughput and hourly price for the workload when choosing a processor.

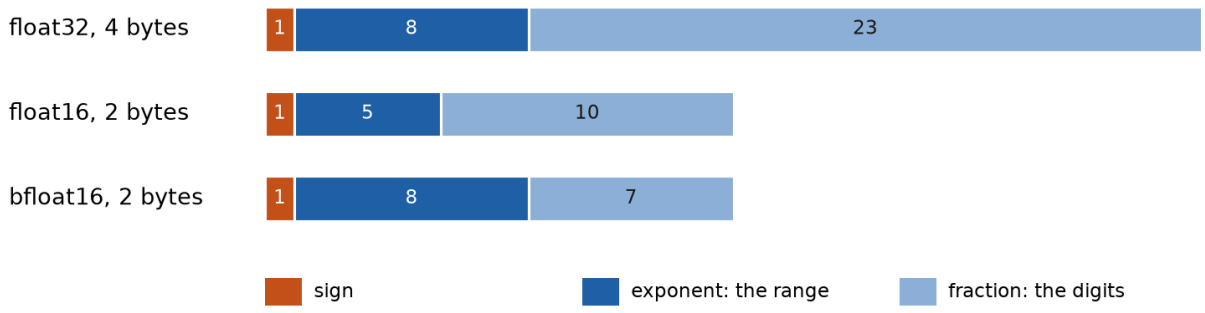
A deeper look 3. Hardware built for the product

Matrix multiplication is a major workload in machine learning. The examples below show how hardware and software are designed for it. The figures come from published data sheets and papers, listed at the end of the section.

5.1 T4 tensor cores and reduced precision

The T4 multiplies	With	Operations a second
32-bit floats: what the class measured	2,560 CUDA cores	8.1 trillion
16-bit floats, with 32-bit sums	320 tensor cores	65 trillion
8-bit integers	320 tensor cores	130 trillion
4-bit integers	320 tensor cores	260 trillion

Besides its 2,560 ordinary cores, the T4 has 320 **tensor cores**, specialized units for multiplying small matrices of 16-bit or smaller numbers. In 16-bit numbers with 32-bit sums the data sheet gives 65 trillion operations a second, eight times the 32-bit peak; in 8-bit and 4-bit integers, twice and four times that again. The class's product used float32 and the ordinary cores, and reached 2.8 trillion; the tensor cores' 16-bit peak is more than twenty times that rate, on the same chip.

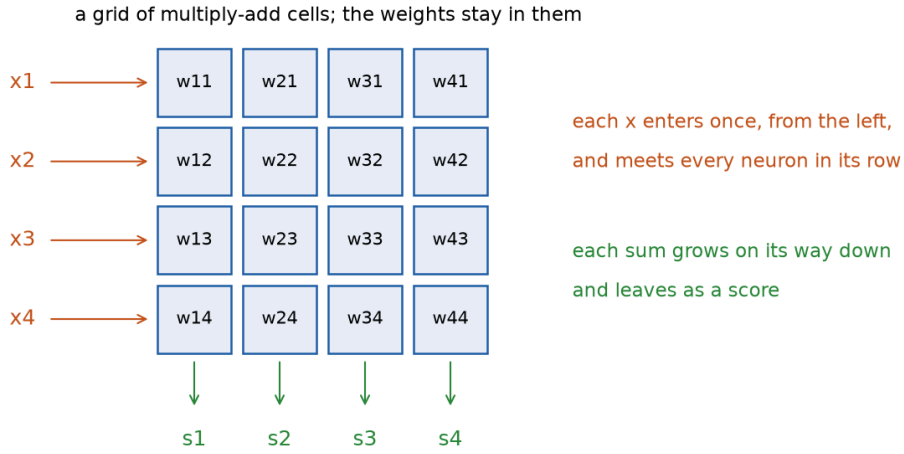


Three formats of a number: the sign, the exponent that sets the range, and the fraction that sets the digits.

A float32 number has a sign, 8 bits of exponent that set its range, and 23 bits of fraction that set its digits. float16 keeps 5 and 10. **bfloat16**, designed by Google Brain for its TPUs, keeps all 8 bits of exponent and only 7 of fraction: the range of float32 in two bytes, with about two decimal digits. This format retains a wide range while using fewer fraction bits, which many neural-network computations can tolerate. Smaller formats reduce both memory traffic and multiplier hardware. Half the bytes means twice the numbers for each byte moved, so twice the operations per byte; and smaller multipliers: on a chip of 2014, a multiplication of two 16-bit floats cost 1.1 picojoules against 3.7 for 32 bits, and a bfloat16 multiplier is about eight times smaller in silicon than a float32 one.

5.2 A chip made for the product: the TPU, and a systolic array

In 2017 Google described its Tensor Processing Unit, a chip specialized for neural-network inference. Its matrix unit has 65,536 multiply-add cells for 8-bit numbers, 92 trillion operations a second at its peak, and 28 mebibytes of memory on the chip itself, managed by the software. On Google's own neural networks it was on average 15 to 30 times as fast as the GPU and the CPU of its day, and did 30 to 80 times as many operations for each watt. Its architecture dedicates much of its hardware to matrix arithmetic.

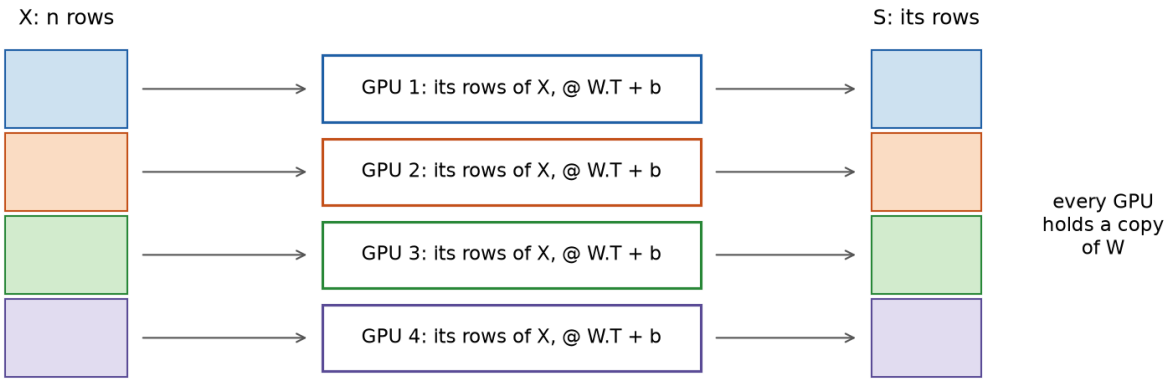


A systolic array: the weights stay in the cells, the coordinates of a point flow through, and the scores leave at the bottom.

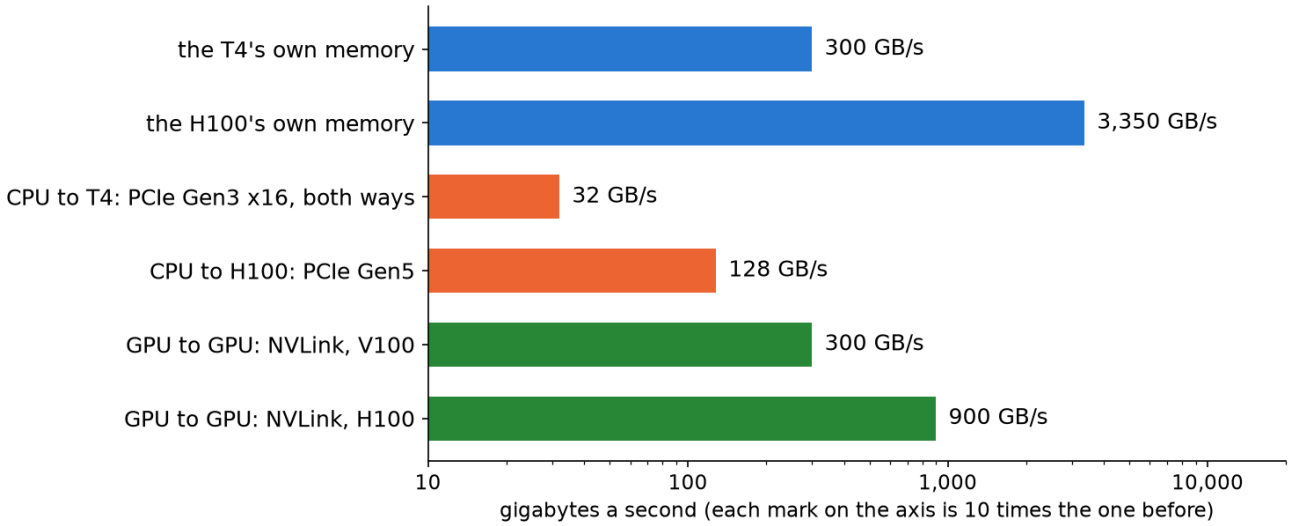
The matrix unit is laid out as a **systolic array**: a grid of multiply-add cells, each holding one weight. The coordinates of a point enter from the left, one row of cells each, and pass from cell to cell; every cell multiplies the number passing through by its weight and adds the product to a running sum coming from above, which leaves the bottom of its column as the score of one neuron. A number fetched from memory is used by every cell of its row before it is dropped, so reuse across a row raises arithmetic intensity. The precise ratio also depends on precision and the traffic for weights and outputs.

5.3 Distributing work across GPUs

The count of GPT-3's training would take a single T4 thousands of years, so the work is split over thousands of GPUs. The simplest way is drawn below: each GPU takes some of the rows of `X`, the points, and computes their rows of `S` with its own copy of `W`. During training, the GPUs combine their computed weight updates and synchronize `W` before the next step. This communication adds a cost that depends on the links between processors.



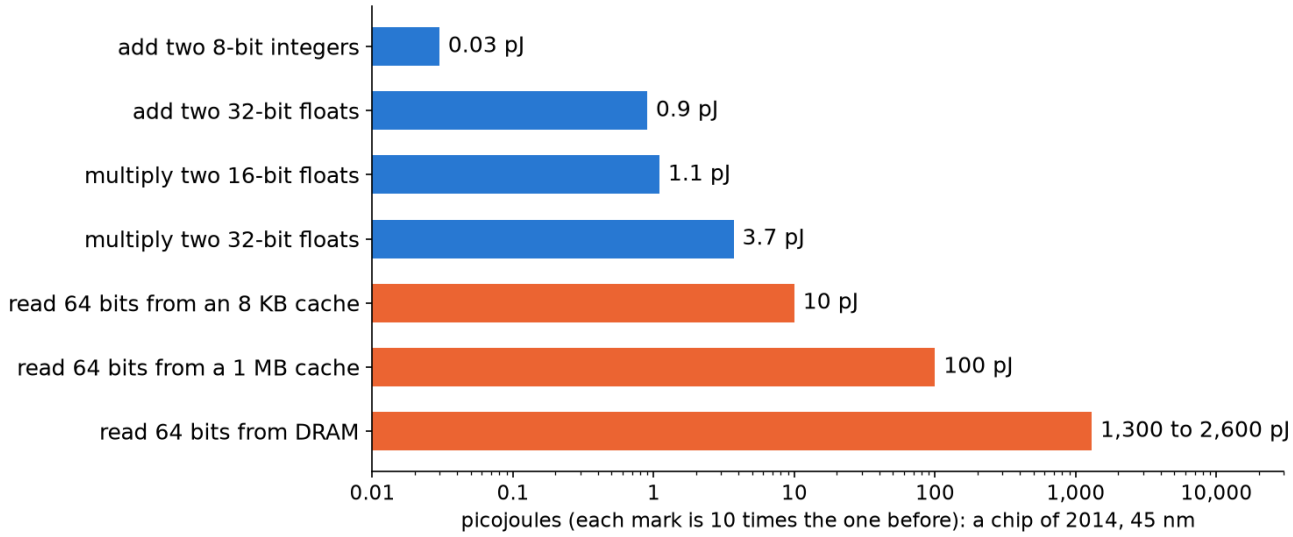
The rows of X split over four GPUs, each with a copy of W: each computes its rows of S.



How many bytes a second each link delivers, by the data sheets of the T4, the V100 and the H100.

A GPU reads its own memory fastest: 300 gigabytes a second on the T4, 3,350 on an H100, the chip of 2022. The PCIe and NVLink figures in the chart are aggregate bidirectional rates, whereas the memory figures follow the data sheets' memory-bandwidth convention; they are not one-way copy rates. To join GPUs to each other NVIDIA built a link of its own, **NVLink**: 300 gigabytes a second for a V100, 900 for an H100, and its switch system joins up to 256 H100s. Between servers the network is narrower still. Large training systems must account for these bandwidth differences when distributing work and data.

5.4 Moving a number costs more than computing with it



The energy of an operation and of a read, on a chip of 2014 (Horowitz, ISSCC 2014).

Horowitz's 2014 estimates for 45-nanometre hardware illustrate the relative energy costs of arithmetic and memory access. Exact values depend on the hardware. A multiplication of two 32-bit floats costs 3.7 picojoules. Reading 64 bits from a small cache costs about three times that, from a large cache about thirty times, and from the main memory, DRAM, hundreds of times. The same paper finds that a general processor spends about 70 picojoules on the overhead of each instruction, which is the interpreter's cost of Recap 2 again, at the level of the hardware. Data movement can dominate energy use. To reduce it, the tensor core, the systolic array and the TPU all keep their numbers next to the multipliers.

5.5 Software and hardware designed for each other

Software written for the chip. Production code typically uses optimized matrix libraries. NumPy hands `X @ W.T` to a **BLAS** library, Basic Linear Algebra Subprograms, such as OpenBLAS or Intel's MKL; PyTorch on an NVIDIA GPU hands it to cuBLAS. Each is written again for each processor: the sizes of its blocks match that chip's caches and registers, its inner loops use that chip's vector instructions, and the work is divided among its cores. The algorithm and its count are those of Recap 1; the rate went from about ten million operations a second in the loops of Python to hundreds of billions on this machine's CPU.

An algorithm written for the memory. The models that write text compare every word of a passage with every other one: the scores QK^T, a softmax of every row, then a second product with a matrix V, three pieces that this unit has met. Written plainly, the matrix of scores, one row and one column for each word, is stored in the GPU's memory and read back, and for a long passage that is most of the time. FlashAttention (Dao and colleagues, 2022) computes the same attention operation block by block in the fast memory on the chip, so that the full matrix of scores is never stored: the

same asymptotic arithmetic complexity with less memory traffic. During the backward pass, it recomputes intermediates instead of loading them from memory, performing extra arithmetic. The paper reports a threefold GPT-2 training speedup for its benchmark.

Models and number formats. Hardware constraints also influence model design and numerical formats. bfloat16 was designed together with Google's TPUs. NVIDIA's tensor cores since 2020 can skip zeros when exactly two of every four weights are zero, **2:4 sparsity**; training methods then keep the accuracy of a network while setting half its weights to zero in that pattern, and the sparse tensor cores do twice the arithmetic a second (Mishra and colleagues, 2021). The H100's data sheet quotes its tensor figures with sparsity, 1,979 trillion 16-bit operations a second, twice its dense ones.

A whole wafer as one chip. Chips are made many at a time on a disc of silicon, a wafer, and cut apart; a GPU is one such rectangle. Cerebras keeps the whole wafer as one chip: its WSE-3, by the company's own product page, is 46,225 square millimetres, 4 trillion transistors and 900,000 cores, with its memory spread over the chip beside the cores, so that the numbers of a layer rarely have to leave it. The design places storage and computation on one wafer to reduce off-chip transfers.

5.6 Operation counts and hardware design

The layer computation uses O(nKm) arithmetic operations. Its running time also depends on interpreter overhead, memory layout, caches, parallel execution, communication, numerical precision, and scheduling. Operation counts and hardware measurements together explain the performance differences seen in this meeting.

Sources of the figures in this section.

- NVIDIA, the data sheets of the T4 (2,560 CUDA cores, 320 tensor cores; 8.1, 65, 130 and 260 trillion operations a second; 16 GB at 300 GB/s; PCIe 32 GB/s; 70 W), the V100 (NVLink 300 GB/s) and the H100 (memory 3.35 TB/s, NVLink 900 GB/s, PCIe 128 GB/s, 1,979 TFLOPS with sparsity, up to 256 GPUs in one switch system).
- N. P. Jouppi et al., *In-Datacenter Performance Analysis of a Tensor Processing Unit*, ISCA 2017.
- M. Horowitz, *Computing's energy problem (and what we can do about it)*, ISSCC 2014, Figure 1.1.9.
- T. Dao, D. Y. Fu, S. Ermon, A. Rudra and C. Ré, *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*, 2022.
- A. Mishra et al., *Accelerating Sparse Deep Neural Networks*, 2021.
- S. Wang and P. Kanwar, *BFloat16: The secret to high performance on Cloud TPUs*, Google Cloud blog, 2019.
- Cerebras, the product page of the WSE-3.
- D. Patterson et al., *Carbon Emissions and Large Neural Network Training*, 2021, for GPT-3; Google Cloud's on-demand prices and the U.S. Energy Information Administration's average price of electricity, as in A deeper look 2.

Closing: Back to the O

The cubic count describes the three-loop algorithm. Other algorithms can reduce the exponent: Strassen's recursion uses seven half-size products, giving $\log_2 7 \approx 2.807$. Dupont et al. (August 2026) report $\omega < 2.371177$. The OpenAI preprint dated October 2, 2026, released October 6, reports $\omega \le 9 / 4 = 2.25$ over the complex numbers.

Precisely, the new statement is $O_\varepsilon(n^{9/4+\varepsilon})$ arithmetic operations for every fixed $\varepsilon > 0$; constants may depend on ε. The preprint leaves the competitive finite size unspecified. A lower arithmetic exponent and a faster measured implementation address different parts of the cost: the amount of work and the rate of executing it.

After the meeting

Each student uploads their own copies of both notebooks to **Upload Your Class Work - Meeting 12**, graded on the eleven cells marked **Your prompt** and **Your thinking**: six in the first notebook (device placement and copies; why the GPU is faster and what a copy costs; a batch twice as large; the prompt for `add_rate`; where the numbers are; how good the prediction was) and five in the second (interpreter overhead and access order; where the order begins to matter; one more number; the prompt for `column_sums_by_rows`; when NumPy cannot choose the order). The group's two discussion posts earn the attention mark for everyone they name; students who do not post earn it by attending.

Keep your copies of the notebooks, with your measurements in them. **Review Quiz 12** on Canvas asks about them: sixteen questions answered with a choice, on the count and a count turned into a time, on devices and copies, on where a number of a matrix is stored, on reading by rows and by columns and at a power of two, on memory-bound and compute-bound, operations per byte and the roofline, and on joules, cents and the energy of moving a number; and one question of a few sentences about your own measurements of Parts 2 and 4 of the first notebook. **Homework 1**, *Vectorization*, has a question on memory-bound work in its Part B. The seven decks of the meeting are linked at the top of these notes and on the meeting's page. The same cost analysis applies to linear-system solvers and to sparse matrices stored by rows or columns.